Logos: A Verified Proof Checker for SMT Solvers in Lean

SMT-LIB Semantics, Specification, and Checker Definitions

1 Introduction

This document summarizes the architecture of Logos, a verified proof checking framework for SMT solvers in Lean. The core motivation behind Logos is to provide Lean-based verification for proof calculi expressed in the Eunoia logical framework.

In detail, Eunoia is a logical framework which allows users to define theories and proof calculi in a *signature*. It is tailored towards SMT solvers, and has a natural compilation to Lean. We do not give a comprehensive description of this compilation in this document.

Logos itself is an executable proof checker. However, its focus is not on performance, but rather to provide a simple and minimal definition of a proof checker in Lean whose correctness can be verified. This allows the correctness of the Eunoia signature (via its compilation to Lean) to be formally verified. In this sense, Logos is a companion to Ethos, an efficient C++-based proof checker for Eunoia.

The core definition of Logos includes a template defining the core proof checker, core definitions for native operations in Lean, as well as definitions of SMT-LIB semantics. The compilation completes these definitions based on the actual proof rules, terms and side conditions. The main correctness proof for Logos is in fact a skeleton which modularizes a theorem to prove to locally show the correctness of each proof rule. The correctness proof of the core checker is fully parameterized based on these theorems, meaning specific instantiations of Logos do not need to prove this.

We overload the term *Logos* specifically as the proof checker generated by compiling the Cooperating Proof Calculus (CPC), the proof calculus used by cvc5, to Lean via this compiler. CPC is formally defined in Eunoia as part of the cvc5 repository. While this version is specific to CPC, note that a similar proof checker can be generated for other Eunoia signatures. We also note that the compilation allows Logos to evolve automatically as the Eunoia signature in cvc5 updates.

In detail, the definition of Logos consists of three parts. First, a formalization of SMT-LIB, defined based on a deep embedding of SMT-LIB terms, types and values. This includes a typing method and a model evaluation method. Second, a deep embedding of Eunoia terms and the definition of satisfiability over them, which relies on a reduction to SMT-LIB terms and types, and the model evaluation method defined in the first layer. Third, an executable proof checker over a deep embedding of Eunoia terms. The latter two layers are meant to be general purpose, although certain non-standard SMT-LIB operators are included in this deep embedding to meet the specific needs of the proof calculus used by Logos. For example, CPC defines several non-standard extensions of SMT-LIB (e.g. sets and sequences), supports non-standard extensions of theories, and defines non-standard internal symbols (i.e. Skolem variables) to define several of its proof rules.

1.1 Deviations from SMT-LIB

The formalization that follows departs from SMT-LIB in three ways, which are different in kind.

First, it admits fewer models than SMT-LIB does in that arrays are interpreted as almost-constant maps. The restriction is stated again at the definition that introduces it. Read on the standard theories alone, every model of this semantics is an SMT-LIB model and the converse does not hold, so unsatisfiability as defined here is implied by unsatisfiability in SMT-LIB and does not imply it: a formula satisfiable in SMT-LIB only by a model this semantics cannot express is treated here as unsatisfiable. It takes quantifiers to reach, and on quantifier-free input the two notions agree.

Second, the semantics interprets sorts and operators that SMT-LIB does not define, including the theories of sets and sequences and the auxiliary operators introduced below. Conformance to SMT-LIB is not defined for these, and what they are measured against is the meaning CPC gives them.

Third, the parser refuses the parametric datatypes SMT-LIB allows whose instances need infinitely many other instances, such as a datatype `N` with a field of sort `(N (Option X))`, rather than interpreting them.

The file `docs/smt-lib-conformance.md` records the same list from the point of view of somebody running the checker, with what each restriction costs.

1.2 An Overview of Core Definitions

Before looking into the definitions, we first provide a high-level overview of the core datatypes and key methods used throughout Logos.

We define the following six types:

- `SmtTerm`, the type of all SMT-LIB terms,
- `SmtType`, the type of all SMT-LIB types,
- `SmtValue`, the type of all SMT-LIB values,
- `SmtModel`, a type corresponding to an SMT-LIB model,
- `Term`, the type of all Eunoia terms and types, and
- `CmdList`, a list of Eunoia commands, i.e. a proof.

SMT-LIB models are a total mapping from identifiers to SMT-LIB values. This mapping provides the interpretation of variables, free constants, and free function symbols. Models are updatable, i.e. to evaluate a quantified formula, we consider all possible updates to the interpretation of the variable that is quantified.

Note that Eunoia does not distinguish terms and types. Instead, we use a single `Term` datatype to represent them. This is for uniformity, as the semantics of Eunoia can be used to manipulate both terms and types. We provide two translation functions from Eunoia terms to SMT-LIB terms and types, as we describe below.

A proof is represented as a list of commands. We will see that executing a command list updates a solver state, which is stack of proven formulas. At the beginning of this list is a list of commands that initialize the solver state with a list of assumed formulas, which we call the *input assumptions* of the proof. A command list is expected to end with a step that concludes false. Full details on commands and checker state are given in Section 4.

We define the following key methods. We break these into three categories. First, methods that define the semantics of SMT-LIB. Second, methods that define the semantics of Eunoia terms based on a reduction to SMT-LIB. Third, the core methods that Logos depends on.

SMT-LIB definition (roughly 2050 LOC):

- `smt_typeof_value : SmtValue -> SmtType`
  - Returns the type of an SMT-LIB value, or none if it is ill-typed.
- `smt_value_canonical : SmtValue -> Bool`
  - Returns whether the given value is *canonical*. For certain types (those that we call *first-class types*, canonical values are syntactically distinct iff they are semantically distinct.
- `smt_type_wf : SmtType -> Bool`
  - Returns whether the given type is well-formed. Our models will range only over well-formed types.
- `smt_typeof : SmtTerm -> SmtType`
  - Returns the type of an SMT-LIB term, or none if it is ill-typed.
- `smt_model_eval : SmtModel -> SmtTerm -> SmtValue`
  - Returns the evaluation of the term in the given model.

Specification definition (roughly 650 LOC):

- `eo_to_smt : Term -> SmtTerm`
  - Given a Eunoia term, this returns the corresponding SMT-LIB term that defines its semantics, or none if none exist.

- `eo_to_smt_type : Term -> SmtType`
  - Given a Eunoia term, this returns the corresponding SMT-LIB type that it corresponds to.
- `eo_satisfiability : Term -> Bool -> Bool`
  - Given a Eunoia term t, this returns true iff t is satisfiable if the second argument is true (resp. unsatisfiable if the second argument is false).

Logos definitions (roughly 8200 LOC):

- `eo_typeof : Term -> Term`
  - Given a Eunoia term, this returns the Eunoia term that corresponds to its type.
- `eo_is_closed : Term -> Term`
  - Given a Eunoia term, this returns the Eunoia term true iff it is a *closed* term, i.e. has no free variables (and the error term `Stuck` on failure).
- `eo_is_refutation : Term -> CmdList -> Bool`
  - `eo_is_refutation` F C returns true iff Logos successfully executes the given command list C, and is a valid, closed proof of false with assumptions F.

Our proof of correctness for Logos will state that under suitable assumptions, `eo_is_refutation` F C implies `eo_satisfiability` F $\bot$. In other words, if we successfully execute Logos given a proof C that assumes F, then indeed F is unsatisfiable. Our trusted core for this specification is thus the definition of `eo_satisfiability`, which is contained in the SMT-LIB and specification layers. The third layer does not need to be trusted. Roughly, the first layer is 2050 lines of Lean, the second layer is 650 lines of Lean (including the shared definition of Eunoia terms), and the third layer is roughly 8200 lines of Lean. In other words, the trusted computing base of Logos is the sum of the first two layers, roughly 2700 lines of Lean.

The executable portion of the Logos checker (the third layer) does *not* depend on the SMT-LIB definition or the specification definition. Instead, they only share the common definition of Eunoia terms (`Term`). Note this means that the SMT-LIB semantics cannot be used as an oracle to define the Logos checker. This is in fact a real restriction, as for instance `smt_model_eval` is a non-computable definition in Lean e.g. due to quantified formulas, and `eo_satisfiability` itself is defined based on quantifying over all SMT-LIB values `SmtValue`. This also properly encapsulates the goal of this verification, namely the Logos definition is intended to be an accurate reflection of the calculus and proof checker in question (e.g. CPC and Ethos), which are reflected in the third layer.

Also, the specification layer above does *not* rely on the Eunoia definitions, e.g. of the Eunoia type system. This is for simplicity, i.e the specification of satisfiability remains as small as possible to minimize the trusted core.

Within the SMT-LIB definition, `smt_model_eval` largely does not depend on e.g. the definitions of the type of terms or values. An exception is that defining the semantics of quantifiers will require a quantified statement over all SMT-LIB values. To guard over a specific type of values, we will call `smt_typeof_value` to guard this statement, as well as the canonicity predicate `smt_value_canonical`.

The definition of the Eunoia type system is encapsulated as a method, `eo_typeof`, which returns the type of a given Eunoia term. Within the definition of Logos, proof rules are essentially untyped manipulations over a single deep embedding of Eunoia types and terms, as encapsulated as `Term`.

A command (in `CmdList`) is either a proof step or an input assumption.[1] A proof checking step in Eunoia succeeds iff the conclusion of that step has type `Bool`, i.e. `eo_typeof` for that term returns the Eunoia term `Bool`. An input assumption command succeeds if it is a *closed* Eunoia term of type `Bool` (via `eo_is_closed`).

We now look in detail at these definitions.

2 SMT-LIB definition

We first discuss SMT-LIB types, terms and values.

A note on presentation: the code shown in figures throughout this document is lightly sanitized with respect to the Lean sources. In particular, we drop the internal name prefixes used by the compiled code (`__smtx_`, `__eo_` and `native_`), we write the native type aliases `Nat`, `Int`, `Bool`, `Char` and `String` for `native_Nat`, `native_Int`,

[1] There are also commands for local assumptions and steps which close them as we describe in Section 4.

```
abbrev Char := Nat
abbrev String := List Char

inductive SmtType : Type where
  | None : SmtType
  | Bool : SmtType
  | Int : SmtType
  | Real : SmtType
  | RegLan : SmtType
  | BitVec : Nat -> SmtType
  | Map : SmtType -> SmtType -> SmtType
  | Set : SmtType -> SmtType
  | Seq : SmtType -> SmtType
  | Char : SmtType
  | Datatype : String -> SmtDatatypeDecl -> SmtType
  | TypeRef : String -> SmtType
  | DtcAppType : SmtType -> SmtType -> SmtType
  | FunType : SmtType -> SmtType -> SmtType
  | USort : Nat -> SmtType
deriving Repr, DecidableEq, Inhabited, Ord

inductive SmtDatatypeDecl : Type where
  | nil : SmtDatatypeDecl
  | cons : String -> SmtDatatype -> SmtDatatypeDecl -> SmtDatatypeDecl
deriving Repr, DecidableEq, Inhabited, Ord

inductive SmtDatatype : Type where
  | null : SmtDatatype
  | sum : SmtDatatypeCons -> SmtDatatype -> SmtDatatype
deriving Repr, DecidableEq, Inhabited, Ord

inductive SmtDatatypeCons : Type where
  | unit : SmtDatatypeCons
  | cons : SmtType -> SmtDatatypeCons -> SmtDatatypeCons
deriving Repr, DecidableEq, Inhabited, Ord

/- Type equality -/
def Teq : SmtType -> SmtType -> Bool
  | x, y => decide (x = y)
```

Figure 1: The SMT type language.

`native_Bool`, `native_Char` and `native_String`, and we write `if`/`then`/`else` for the method `native_ite`. We additionally use a small number of friendlier names, e.g. `Cmd` and `CmdList` for the checker types `CCmd` and `CCmdList`. Beyond renaming, figures may inline or abbreviate helper definitions, reorder declarations, and elide cases (indicated by ellipses); the presentation is otherwise faithful to the code.

2.1 Types

Figure 1 gives the definition of SMT-LIB types in our embedding. This includes an error value (`None`) denoting not a type,[2] the standard atomic SMT-LIB types of Booleans (`Bool`), integers (`Int`), reals (`Real`) and regular languages (`RegLan`). Bitvectors (`BitVec`) are indexed by a natural number denoting the bitwidth. Note that in contrast to SMT-LIB, our semantics permit the bitvectors of size zero.

We then define three composite types:

- The type of maps (`Map`) will be used to reason about SMT-LIB arrays. Note that this type will have a minor difference with the official type of arrays in SMT-LIB, namely, it will interpreted as admitting only values that are almost constant maps.
- The type of finite sets (`Set`).
- The type of sequences (`Seq`). Note that for uniformity, SMT-LIB strings are represented as a sequence of characters, where the character type (`Char`) is another atomic type constructor and has a finite range of values determined by the range of Unicode in SMT-LIB (values 0 to 196607).

Note the latter two types are not in the SMT-LIB standard.

[2]One could alternatively use an `Option` type instead of allowing `None` as a first-class constructor. Throughout, we prefer using an error elements of types as opposed to option types to keep the control flow simple.

The next three types pertain to SMT-LIB datatypes. The first (`Datatype`) denotes a (possibly recursive) datatype type. In this definition, the datatype carries its name (via a string[3]) and a datatype *declaration* inside the abstract syntax tree. In particular, the first argument of the datatype denotes the name of that datatype. The second argument (a `SmtDatatypeDecl`) is a declaration block: a list of (name, datatype) pairs that contains the definition of the datatype in question along with the definitions of all datatypes it is mutually recursive with. A datatype definition (`SmtDatatype`) is a list of constructors (`SmtDatatypeCons`), where `sum` adds a constructor to this list, and `null` terminates its list of constructors. A constructor itself is a list of types, indicating the types of its children, or fields. References to a datatype of the declaration block are indicated by type references (constructor `TypeRef`), which takes the name of the datatype that it references. In other words, a type `(TypeRef s)` occurring as a field of a constructor refers to the datatype declared under name `s` in the enclosing declaration block. For details on this, see Section 2.2.

The next type (`DtcAppType`) is the type of partially applied constructors. It is a function-like type that takes its argument and return types. The type of partially applied constructors is intentionally distinct from function types (`FunType`), which also takes its argument and return types. Both these types are curried, for instance the function type expecting two integers and returning a Boolean is `(FunType Int (FunType Int Bool))`. These two types are distinct for several reasons, the primary being that SMT values of type `DtcAppType` type can be "applied" to other values to construct new values. As we will describe in Section 2.4, this reflects the fact that datatype types are given Herbrand interpretations, i.e. their values are term-like. In contrast, function values cannot be applied to values to construct new values.

The final constructor in this list denotes uninterpreted sorts (`USort`), which are indexed by a natural number identifier.

All types have the Lean properties of being representable (`Repr`), having decidable equality (`DecidableEq`), are inhabited (`Inhabited`) and orderable (`Ord`). Due to decidable equality, we can define (syntactic) equality on types, as `Teq`.

2.2 Datatypes

This section presents how SMT-LIB datatype types are represented in our embedding. At a high level, a datatype type carries its full *declaration*: the type `(Datatype s dd)` packages the name `s` of the datatype together with a declaration block `dd` that maps names to datatype definitions. All datatypes of a mutually recursive group are declared together in a single block, and recursive references within the block are made by name via `TypeRef`.
For example, consider the SMT-LIB datatype for lists of integers:

```
(declare-datatype List ((listc (head Int) (tail List)) (listn)))
```

The datatype type for list is `(Datatype "List" DD)`, where

```
DD := (SmtDatatypeDecl.cons "List" DList SmtDatatypeDecl.nil)
DList := (sum (cons Int (cons (TypeRef "List") unit)) (sum unit null))
```

Here, `DD` is a declaration block declaring a single datatype under the name `"List"`, whose definition `DList` is a list containing two datatype constructors. The first (corresponding to `listc`) contains the field types `Int` and `(TypeRef "List")`. Intuitively, the latter can be thought of as a name bound by the enclosing declaration block, i.e. it specifies that the second child of this constructor is itself a list. The second constructor in this list (corresponding to `listn`) is given by `unit`, marking it as a constructor taking no arguments.

This definition also permits mutually recursive datatypes, in which case the declaration block contains one entry per datatype of the mutually recursive group. For example, consider the SMT-LIB declaration:

```
(declare-datatypes ((Tree 0) (TreeList 0))
  (((node (children TreeList)) (leaf))
   ((tlistc (head Tree) (tail TreeList)) (tlistn))))
```

This generates a single declaration block `DD` that is shared by the two datatypes:

```
DD := (cons "Tree" DTree (cons "TreeList" DTreeList nil))
DTree := (sum (cons (TypeRef "TreeList") unit) (sum unit null))
DTreeList := (sum (cons (TypeRef "Tree") (cons (TypeRef "TreeList") unit)) (sum unit null))
```

The datatype types for the two datatypes are `(Datatype "Tree" DD)` and `(Datatype "TreeList" DD)` respectively; note that they differ only in their name component and share the same declaration block. Within `DTree`, the first

[3]Throughout, we define a string to be a list of natural numbers and do not use native Lean strings. This is to avoid complications in the definition of Lean strings which take peculiarities with Unicode into account.

constructor (corresponding to `node`) has a single field that references `TreeList`, and within `DTreeList`, the first constructor (corresponding to `tlistc`) contains two type references, to `Tree` and to `TreeList`. In contrast to representations based on mu-types, no datatype definition is ever inlined beneath another: all recursive references, whether to the datatype itself or to a sibling of its group, are by name into the shared declaration block.

Note that the names of constructors and selectors are not given in the representation of datatypes. Instead constructors are referenced by natural number indices. For example, in `DList` above, 0 will refer to `listc` and 1 will refer to `listn`. Similarly, we will use two natural numbers to reference selectors, e.g. in `DList` (0, 0) will refer to `head` and (0,1) to `tail`. Details on this will be given when values and terms are introduced (Sections 2.4 and 2.8).

This definition permits four kinds of datatypes which we wish to exclude from input formulas. First, a type may contain a type reference that is not in the scope of a declaration block that declares the name it references. Examples of this kind of datatype are the following:

```
(TypeRef "List")
(Map Int (TypeRef "List"))
(Datatype "Tree" (cons "Tree" (sum (cons (TypeRef "List") unit) null) nil))
```

In the first two examples the reference does not occur as a constructor field of any declaration block at all; in the third, the constructor of `"Tree"` references the name `"List"`, which the declaration block does not declare. In this case, we say that the type is not well-formed. Our formalization in SMT-LIB will later define a well-formedness predicate that defines a subset of types that are considered well-formed.

Second, a datatype itself may be uninhabited, i.e. there are no `SmtValue` that have that type. Examples of this kind of datatype are the following:

```
(Datatype "no-wf" (cons "no-wf" (sum (cons Int (cons (TypeRef "no-wf") unit)) null) nil))
(Datatype "empty" (cons "empty" null nil))
(Datatype "mutual-no-wf"
  (cons "mutual-no-wf" (sum (cons (TypeRef "aux") unit) null)
  (cons "aux" (sum (cons (TypeRef "mutual-no-wf") unit) null) nil)))
```

In the first example, a datatype whose only constructor contains a recursive reference to itself is uninhabited. Other examples include a datatype with no constructors (the second example), or a mutually recursive group where every way of constructing a value eventually requires a value of the datatype being defined (the third example, where the constructors of `"mutual-no-wf"` and `"aux"` each reference the other and neither has a base case).

Third, a declaration block may declare the same name more than once. An example of such a datatype is:

```
(Datatype "aliased" (cons "aliased" d1 (cons "aliased" d2 nil)))
```

Above, the declaration block contains two entries with the name `"aliased"` (for some constructor lists `d1` and `d2`). Name lookup would silently resolve references to the first entry. By convention, we consider such declarations to be ill-formed.

Fourth, a datatype may contain *nested recursion*, that is, references to the datatype being defined beneath foreign types. An example of such a datatype is:

```
(Datatype "nested-rec"
  (cons "nested-rec" (sum (cons (Map Int (TypeRef "nested-rec")) unit) null) nil))
```

Here, this datatype contains a single constructor having a single field that is of type `(Map Int (TypeRef "nested-rec"))`, where note that the element sort of this map is a recursive reference to the datatype being defined. Such datatypes are generally not supported by SMT solvers, and moreover are not allowed by the SMT-LIB standard. In our definition, we will consider these datatypes to be ill-formed.

2.2.1 Datatype Lookup and Resolution

The two common operations on datatypes in this representation are *lookup*, which retrieves the definition of a datatype from a declaration block by name, and *resolution*, which replaces the type references occurring in a definition by self-contained datatype types. These operations are used when giving type rules for datatype constructors, selectors and other builtin datatype operations. Their definitions are given by the methods `smt_dd_lookup` and `smt_dt_resolve` in Figure 2.

In detail, the method `(smt_dd_lookup s dd)` scans the declaration block `dd` for the first entry declared under the name `s` and returns its list of constructors, returning the empty constructor list (`SmtDatatype.null`) when the name is not declared. The companion method `(smt_dd_has_dt s dd)` returns whether `dd` declares the name `s`; it will serve as the "in scope" check for type references throughout the remaining definitions.

```
def smt_dtc_resolve : SmtDatatypeCons -> SmtDatatypeDecl -> SmtDatatypeCons
  | (SmtDatatypeCons.cons (SmtType.TypeRef s) c), dd => (SmtDatatypeCons.cons (SmtType.Datatype s dd) (smt_dtc_resolve c dd))
  | (SmtDatatypeCons.cons T c), dd => (SmtDatatypeCons.cons T (smt_dtc_resolve c dd))
  | SmtDatatypeCons.unit, dd => SmtDatatypeCons.unit

def smt_dt_resolve : SmtDatatype -> SmtDatatypeDecl -> SmtDatatype
  | (SmtDatatype.sum c d), dd => (SmtDatatype.sum (smt_dtc_resolve c dd) (smt_dt_resolve d dd))
  | SmtDatatype.null, dd => SmtDatatype.null

def smt_dd_lookup (s : String) : SmtDatatypeDecl -> SmtDatatype
  | (SmtDatatypeDecl.cons s2 d dd) => (if (s = s2) then d else (smt_dd_lookup s dd))
  | SmtDatatypeDecl.nil => SmtDatatype.null

def smt_dd_has_dt (s : String) : SmtDatatypeDecl -> Bool
  | (SmtDatatypeDecl.cons s2 d dd) => ((s = s2) || (smt_dd_has_dt s dd))
  | SmtDatatypeDecl.nil => false
```

Figure 2: Datatype declaration lookup and reference resolution.

The method `(smt_dt_resolve d dd)` takes a constructor list `d` and a declaration block `dd`, and replaces each field of the form `(SmtType.TypeRef s)` occurring (directly) in a constructor of `d` with the self-contained datatype type `(SmtType.Datatype s dd)`. The traversal is performed by `smt_dt_resolve` over the constructor list and by `smt_dtc_resolve` over the fields of each constructor. All other field types are left unchanged. In particular, resolution does not traverse composite field types like maps, sequences and sets, so type references occurring beneath foreign types are not replaced. This behavior is arbitrary, as we do not consider datatypes with nested recursion to be well-formed.

As an example, consider the following mutually recursive datatypes, in smt2:

```
(declare-datatypes ((A 0) (B 0))
  (((mkA (b B)))
   ((mkB (a A)) (leaf))))
```

Following the conventions above, the datatype type for `A` is `(Datatype "A" DD)`, where

```
DD := (cons "A" DA (cons "B" DB nil))
DA := (sum (cons (TypeRef "B") unit) null)
DB := (sum (cons (TypeRef "A") unit) (sum unit null))
```

The constructor list of `A` is obtained by `(smt_dd_lookup "A" DD)`, which returns `DA`. Resolving it, `(smt_dt_resolve DA DD)` returns

```
(sum (cons (Datatype "B" DD) unit) null)
```

i.e. the field of the constructor `mkA` is now the standalone type `(Datatype "B" DD)`, which carries the same declaration block `DD`.

Overall the purpose of resolution is to make the field types of a datatype self-contained: after looking up a datatype's constructor list and resolving it against its own declaration block, every field is an ordinary `SmtType` whose meaning does not depend on the position at which it occurs. The composition `(smt_dt_resolve (smt_dd_lookup s dd) dd)` is the idiom used to give type rules for datatype values and terms, as well as for defining the construction of canonical values for datatype types. Note that, unlike one-step unfolding of mu-types, resolution never copies or rewrites the declaration block itself, so no "lifting" of nested definitions is required.

2.3 Models

Before giving the definition of SMT-LIB values (`SmtValue`), we first give a definition of `SmtModel`. A model consists of two mappings and a cardinality function. The first (`values`) is a mapping from keys (as defined via the structure `SmtModelKey`) to values. The second (`nativeFuns`) gives a native (unary) Lean function for each function identifier and pair of domain and range types. The third (`usortDom`) gives the domain of each uninterpreted sort, as described in Section 2.4. A model key consists of three things: (1) whether the key is for a variable (resp. free constant), (2) a string identifier for the variable or constant, and (3) the type of the variable or constant.

We will be only interested in models that have several properties, namely that they are total mappings (over both value and function mappings). We also require that they are *well-typed*, namely, that they map identifiers having a given type to values with that type (for the mapping `values`). For the mapping `nativeFuns`, at types `T` and `U` with `(FunType T U)` well-formed, we require that for all input values of type T, the Lean function successfully returns a

```
structure SmtModelKey where
  isVar : Bool
  name : String
  ty : SmtType
deriving Repr, DecidableEq, Inhabited

abbrev ModelId := Real

structure SmtModel where
  values : SmtModelKey -> SmtValue
  nativeFuns : ModelId -> SmtType -> SmtType -> (SmtValue -> SmtValue)
  usortDom : Nat -> Set ModelId
deriving Inhabited

def model_push (M : SmtModel) (s : String) (T : SmtType) (v : SmtValue) : SmtModel :=
  { M with values := fun k =>
      if k = { isVar := true, name := s, ty := T } then
        v
      else
        M.values k }
```

Figure 3: Models and update helpers.

value of type U. All other indices in either map are not used. We will additionally impose a property of *canonicity* on the values that are mapped to by an SMT-LIB model, which we will describe later.

2.4 Values

Figure 4 contains the complete definition of SMT-LIB values, which roughly correspond one-to-one with SMT-LIB types. It is important to note that the definition of this datatype does not contain any nested recursion. In particular, we do not embed e.g. maps, sequences or sets over values as subfields of this datatype. Instead, these are reified as data. This datatype permits both values that are ill-typed as well as multiple values that are semantically equivalent. A subset of `SmtValue` will be considered well-typed values, and a further subset of these will be considered *canonical* values.

Like types, we use an error constructor `NotValue` to denote a failure in model evaluation. We then have the atomic constructors for Booleans, integers and reals. The constructor for the first takes native Lean Booleans and integers respectively. The values of type Real (`Real`) are the real numbers, given by Mathlib's `Real`. Only the literals of the term language are rational, and each denotes its image in the reals. For bitvectors, we use a constructor (called `Binary`) that takes two integers as input denoting the bitwidth and the value. We assume that the bitwidth w is non-negative and the value is in the range $[0, 2^w)$. Note that we do not make use of builtin naturals (Nat) or native Lean bit-vectors here. We chose not to use Nat to avoid having to overload many arithmetic operations when defining SMT-LIB evaluations (since SMT-LIB does not define Nat). We chose not to use native bit-vectors to avoid dependently typed programming.[4]

Map values are given by the constructor `Map`, which takes an auxiliary datatype, `SmtMap` as an argument. The SMT map datatype itself is defined as a list of index, element pairs (via `SmtMap.cons`) which indicate the *stores* of the map value, followed with a default value (via `SmtMap.default`). A default value stores the index type and the element that is stored at every index. In other words, `(cons (Numeral 0) (Numeral 1) (default Int (Numeral 2)))` represents the map where integer 0 is mapped to 1, and all other integers are mapped to 2. Maps will be used to represent values of the SMT-LIB type Array. Note that this is a slight divergence from the official semantics, which does not insist on a concrete interpretation of this type. This implies that our semantics will insist that Array is interpreted as almost-constant maps. Map values may contain redundant entries, for instance `(cons (Numeral 0) (Numeral 1) (cons (Numeral 0) (Numeral 3) (default Int (Numeral 2))))` represents the same map as the previous one, where the index 0 was "overwritten" at index 0 by the value 1. Similarly, the store in `(cons (Numeral 0) (Numeral 1) (default Int (Numeral 0)))` is also redundant. In general, this will require us to define a notion of canonicty of array values, which we define later. Furthermore, note that a constructed map may not be typeable, e.g. `(const (Numeral 0) (Numeral 1) (default Int (Boolean true)))` stores integer indices but has a Boolean value as its default. We will see later that these values will be considered ill-typed.

Function values are represented by the constructor `Fun`, which takes three arguments (an identifier and two types). These arguments together are an identifier that is used to lookup the behavior of the function value in the model, as a native Lean function `SmtValue -> SmtValue`. In detail, the first argument r is an identifier of type `ModelId`, which abbreviates `Real`, the second argument T is its domain type, and the third argument U is its range type.

[4]This decision was made since Isabelle is an alternative target for compilation from Eunoia.

```
inductive SmtValue : Type where
  | NotValue : SmtValue
  | Boolean : Bool -> SmtValue
  | Numeral : Int -> SmtValue
  | Real : Real -> SmtValue
  | Binary : Int -> Int -> SmtValue
  | Map : SmtMap -> SmtValue
  | Fun : ModelId -> SmtType -> SmtType -> SmtValue
  | Set : SmtMap -> SmtValue
  | Seq : SmtSeq -> SmtValue
  | Char : Char -> SmtValue
  | RegLan : SmtRegLan -> SmtValue
  | DtCons : String -> SmtDatatypeDecl -> Nat -> SmtValue
  | Apply : SmtValue -> SmtValue -> SmtValue
  | UValue : Nat -> ModelId -> SmtValue
deriving Repr, DecidableEq, Inhabited, Ord

inductive SmtMap : Type where
  | cons : SmtValue -> SmtValue -> SmtMap -> SmtMap
  | default : SmtType -> SmtValue -> SmtMap
deriving Repr, DecidableEq, Inhabited, Ord

inductive SmtSeq : Type where
  | cons : SmtValue -> SmtSeq -> SmtSeq
  | empty : SmtType -> SmtSeq
deriving Repr, DecidableEq, Inhabited, Ord

inductive SmtRegLan : Type where
  | empty : SmtRegLan
  | epsilon : SmtRegLan
  | char : SmtValue -> SmtRegLan
  | range : SmtValue -> SmtValue -> SmtRegLan
  | allchar : SmtRegLan
  | concat : SmtRegLan -> SmtRegLan -> SmtRegLan
  | union : SmtRegLan -> SmtRegLan -> SmtRegLan
  | inter : SmtRegLan -> SmtRegLan -> SmtRegLan
  | star : SmtRegLan -> SmtRegLan
  | comp : SmtRegLan -> SmtRegLan
deriving Repr, DecidableEq, Inhabited, Ord

def veq : SmtValue -> SmtValue -> Bool
  | x, y => decide (x = y)

def vcmp (v1 : SmtValue) (v2 : SmtValue) : Bool :=
  match compare v1 v2 with
  | Ordering.lt => true
  | _ => false
```

Figure 4: SMT-LIB values, and auxiliary definitions of maps, sequences, and regular languages.

Given a model M, the behavior of a function value is the Lean function `(M.nativeFuns r T U)`. In other words, function values use a level of indirection for their definition. Since functions of several arguments are curried, a partial application is such a value too. Identifiers are real numbers so that there are enough of them: a model can interpret a binary function on `Real` as $<$, for instance, using a different identifier for each real. As a downside, this means that we cannot do computations over function values, nor can we define equality over function values in a manner that preserves extensionality. Note that two syntactically distinct function values may be mapped to extensionally identical Lean functions by the model. However, this ensures that our definition of the domain of functions is accurate with respect to SMT-LIB semantics of uninterpreted functions.

Set values are represented by constructor `Set`, which reuses the SMT map datatype that was used for the map type. Intuitively, this map maps all elements to whether they occur in the set. Like map values, there may be many ways of representing the same set. Note that for *finite* sets, we will insist that the default value given to the set is false, i.e. a set is a finite list of elements (storing true), and the default says that no further elements are in the set.

Sequence values are represented by constructor `Seq`, which uses a different auxiliary datatype (`SmtSeq`) to represent them. This datatype is a simply a list of `SmtValue`, where note that the empty sequence carries the type of its elements. Like maps and sets, it is possible to construct a ill-typed sequence by e.g. the concatenation `(cons (Int 0) (cons (Bool true) (empty Int)))`. Recall that strings are defined as a sequence of chars. The next constructor (`Char`) defines elements of this sort, which are expected to be natural numbers within the range of characters specified by SMT-LIB (0 to 196607).

Regular language values (`RegLan`) use an auxiliary datatype (`SmtRegLan`) in their definition. Note that the base elements of regular languages (the arguments of `char` and `range`) are `SmtValue`s rather than characters. This allows regular expression operations to be defined uniformly over the same unpacked sequence representation used by the sequence operations; well-formed regular languages carry only valid character values (`SmtValue.Char`) as base elements. This representation is not canonical: multiple distinct ASTs may denote the same regular language. Consequently, later we will see that equality on regular expression values will be defined extensionally, i.e., two values are equal iff they accept the same set of strings.

The next two constructors (`DtCons` and `Apply`) are used to denote datatype values. In particular, the constructor `DtCons` takes the same first two arguments as a datatype type: the name of its datatype and the declaration block that defines its structure. The third argument is a natural number that denotes the index of the constructor. Recall the declaration block `DD` from the previous section, which corresponded to the SMT-LIB datatype

```
(declare-datatype List ((listc (head Int) (tail List)) (listn)))
```

and had the encoding:

```
DD := (SmtDatatypeDecl.cons "List" DList SmtDatatypeDecl.nil)
DList := (sum (cons Int (cons (TypeRef "List") unit)) (sum unit null))
```

Values that denote the constructors of this datatype are given by `(DtCons "List" DD 0)` and `(DtCons "List" DD 1)`. The former corresponds to `listc` and the latter corresponds to `listn`. The value constructor `Apply` is used to construct applications of datatype constructors to values. It is a higher-order application, taking the head (a partially applied constructor value) and an argument to apply to. For instance, the value corresponding to the singleton list containing the integer 7 is:

```
(Apply (Apply (DtCons "List" DD 0) (Numeral 7)) (DtCons "List" DD 1))
```

Applications in this style are curried. Our typing rules on SMT-LIB values will restrict the `Apply` value constructor to only allow partially applied datatype constructors as the head, and not for instance functions. Again, this is because datatypes are given Herbrand interpretations.

Values of uninterpreted sort are represented by constructor `UValue`, which take a natural number index and an identifier. The first corresponds to the identifier of their uninterpreted sort, where recall uninterpreted sorts are of the form `(USort n)` for natural number n. The second is the identifier for the value, of type `ModelId`, where we assume that `(UValue n i)` and `(UValue n j)` are distinct when i and j are distinct. The domain of each uninterpreted sort is chosen by the model: `(M.usortDom n)` is a set of identifiers, and the values of `(USort n)` are exactly `(UValue n r)` for r in that set, so its size is the cardinality of the sort. The definition of a well-formed model requires it to contain 0, so that the sort is nonempty and has a default value, and to be unbounded if it is infinite. Since identifiers are only labels, neither condition rules out a cardinality: every nonzero cardinality up to the size of `Real` is available. SMT-LIB has no upper limit, but as far as we can tell a larger sort can only be forced through the full function space of an array sort, which is already the array divergence. The restriction to the values of the sort is enforced by canonicity, described below.

Values have the same type class properties as types, and can be compared with `veq`. We will additionally require

```
def smt_typeof_guard (T : SmtType) (U : SmtType) : SmtType :=
  (if (Teq T SmtType.None) then SmtType.None else U)

def smt_typeof_map_value : SmtMap -> SmtType
  | (SmtMap.cons i e m) =>
    let _v0 := (smt_typeof_map_value m)
    (if (Teq (SmtType.Map (smt_typeof_value i) (smt_typeof_value e)) _v0) then _v0 else SmtType.None)
  | (SmtMap.default T e) => (SmtType.Map T (smt_typeof_value e))

def smt_map_to_set_type : SmtType -> SmtType
  | (SmtType.Map T SmtType.Bool) => (SmtType.Set T)
  | T => SmtType.None

def smt_typeof_seq_value : SmtSeq -> SmtType
  | (SmtSeq.cons v vs) =>
    let _v0 := (smt_typeof_seq_value vs)
    (if (Teq (SmtType.Seq (smt_typeof_value v)) _v0) then _v0 else SmtType.None)
  | (SmtSeq.empty T) => (SmtType.Seq T)

def smt_typeof_dt_cons_value_rec (T : SmtType) : SmtDatatype -> Nat -> SmtType
  | (SmtDatatype.sum SmtDatatypeCons.unit d), Nat.zero => T
  | (SmtDatatype.sum (SmtDatatypeCons.cons U c) d), Nat.zero =>
    (SmtType.DtcAppType U (smt_typeof_dt_cons_value_rec T (SmtDatatype.sum c d) Nat.zero))
  | (SmtDatatype.sum c d), (Nat.succ n) => (smt_typeof_dt_cons_value_rec T d n)
  | d, n => SmtType.None

def smt_typeof_apply_value : SmtType -> SmtType -> SmtType
  | (SmtType.DtcAppType T U), V => (smt_typeof_guard T (if (Teq T V) then U else SmtType.None))
  | T, U => SmtType.None

...

def smt_typeof_value : SmtValue -> SmtType
  | (SmtValue.Boolean b) => SmtType.Bool
  | (SmtValue.Binary w n) =>
      if 0 <= w ∧ n = n % (2 ^ w) then SmtType.BitVec (Int.toNat w) else SmtType.None
  | (SmtValue.Char c) => (if (c < 196608) then SmtType.Char else SmtType.None)
  | (SmtValue.Map m) => (smt_typeof_map_value m)
  | (SmtValue.Set m) => (smt_map_to_set_type (smt_typeof_map_value m))
  | (SmtValue.Seq ss) => (smt_typeof_seq_value ss)
  | (SmtValue.DtCons s dd i) => (smt_typeof_dt_cons_value_rec (SmtType.Datatype s dd) (smt_dt_resolve (smt_dd_lookup s dd) dd) i)
  | (SmtValue.Apply f v) => (smt_typeof_apply_value (smt_typeof_value f) (smt_typeof_value v))
  | (SmtValue.UValue i e) => (SmtType.USort i)
  | ... => ...
```

Figure 5: Types for SMT values.

an (arbitrary strict total) ordering over values, which we define as `vcmp`. Note that this definition requires an instantiation over the native Lean types that are used within this datatype. In practice, this required us to define a strict total ordering over the reals for this purpose; the ordering for all other types were automatic.

Recall that the primary purpose of values is to represent the output of model evaluation. We will impose certain restrictions on SMT-LIB values. In particular, later we define *canonical* as a property of (a subclass of) SMT-LIB values with the following behavior: two canonical values that are syntactically distinct are guaranteed to be semantically distinct. Canonicity, however, will not apply to values over all types. In particular, we will not define a notion of canonicity (in the above sense) for functions and regular expression values. This will limit their usage when considering the space of well-formed types we will consider.

2.5 Types for SMT-LIB Values

In this section, we present type rules for SMT-LIB values and terms. Type rules will be used to define the semantics of quantified formulas and our definition of SMT-LIB models.

Our definition of `smt_typeof_value` is given in Figure 5. We present a representative subset of value cases.

This method first contains the obvious base cases for e.g. Boolean values. For bit-vector values (`Binary`), we consider only values that specify well-formed and canonical bit-vector values to be well-typed. In particular, this means their bitwidth must be non-negative and their value must be in the interval $[0, 2^w)$; otherwise they are ill-typed (i.e. have type `SmtType.None`). Similarly, characters are ill-typed if their value falls outside the legal range of characters as defined by SMT-LIB ($[0, 196608)$).

Recall that maps and set values share a common payload (`SmtMap`). We define a method computing the type for this datatype (`smt_typeof_map_value`). In particular, note that a map is considered ill-typed if any `cons` term has index and element type that does not match the tail. For sets, we cast the type for the returned map to a set type

```
def smt_field_type_bounded (uc : Nat -> Nat) (u : Bool) : SmtType -> SmtDatatypeDecl -> Bool
  | (SmtType.TypeRef s), ddB => (smt_dd_has_dt s ddB)
  | T, ddB => (smt_type_bounded uc u T)

def smt_datatype_cons_bounded (uc : Nat -> Nat) (u : Bool) : SmtDatatypeCons -> SmtDatatypeDecl -> Bool
  | SmtDatatypeCons.unit, ddB => true
  | (SmtDatatypeCons.cons T c), ddB => ((smt_field_type_bounded uc u T ddB) && (smt_datatype_cons_bounded uc u c ddB))

def smt_datatype_bounded (uc : Nat -> Nat) (u : Bool) : SmtDatatype -> SmtDatatypeDecl -> Bool
  | (SmtDatatype.sum c SmtDatatype.null), ddB => (smt_datatype_cons_bounded uc u c ddB)
  | (SmtDatatype.sum c dF), ddB => (!u && ((smt_datatype_cons_bounded uc u c ddB) && (smt_datatype_bounded uc u dF ddB)))
  | dF, ddB => !u

def smt_datatype_decl_bounded_step (uc : Nat -> Nat) (u : Bool) : SmtDatatypeDecl -> SmtDatatypeDecl -> SmtDatatypeDecl
  | (SmtDatatypeDecl.cons sF dF ddR), ddB => (smt_datatype_decl_bounded_step uc u ddR (if (!(smt_dd_has_dt sF ddB) && (
      smt_datatype_bounded uc u dF ddB)) then (SmtDatatypeDecl.cons sF dF ddB) else ddB))
  | SmtDatatypeDecl.nil, ddB => ddB

def smt_datatype_decl_bounded (uc : Nat -> Nat) (u : Bool) : SmtDatatypeDecl -> SmtDatatypeDecl -> SmtDatatypeDecl -> SmtDatatypeDecl
  | (SmtDatatypeDecl.cons sC dC ddC), dd, ddB => (smt_datatype_decl_bounded uc u ddC dd (smt_datatype_decl_bounded_step uc u dd ddB))
  | SmtDatatypeDecl.nil, dd, ddB => ddB

def smt_type_bounded (uc : Nat -> Nat) (u : Bool) : SmtType -> Bool
  | SmtType.Bool => !u
  | (SmtType.USort i) => (if u then (uc i = 1) else (uc i != 0))
  | (SmtType.BitVec w) => (!u || (w = 0))
  | SmtType.Char => !u
  | (SmtType.Datatype s dd) => (smt_dd_has_dt s (smt_datatype_decl_bounded uc u dd dd SmtDatatypeDecl.nil))
  | (SmtType.Map T U) => ((smt_type_bounded uc true U) || (!u && ((smt_type_bounded uc u T) && (smt_type_bounded uc u U))))
  | (SmtType.Set T) => (!u && (smt_type_bounded uc u T))
  | T => false

def smt_is_unit_type (uc : Nat -> Nat) (T : SmtType) : Bool :=
  (smt_type_bounded uc true T)

def smt_is_finite_type (uc : Nat -> Nat) (T : SmtType) : Bool :=
  (smt_type_bounded uc false T)
```

Figure 6: Unit and finite SMT type predicates.

(method `smt_map_to_set_type`), noting that the element type of the map must be `Bool`.

Sequences values are typed similarly using a recursive method `smt_typeof_seq_value`. Like maps and sets, this may return that the sequence is ill-typed if any element does not match the element type of its tail.

Datatype constructor values use a recursive method to compute their type as well. For a constructor value `(DtCons s dd i)`, we first obtain the constructor list of the datatype by looking up the name `s` in the declaration block and resolving its type references, i.e. `(smt_dt_resolve (smt_dd_lookup s dd) dd)` (recall Section 2.2.1), so that all field types are self-contained. The method `smt_typeof_dt_cons_value_rec` then takes the return type of the constructor (`(Datatype s dd)`), the resolved constructor list, and the index `i` of the constructor in question. If `i` is not zero, we recurse to the next constructor. If `i` is zero, then for each field type, we prepend an argument type via the partially applied constructor type `SmtType.DtcAppType`. Again, recall the datatype declaration `DD` for `List`: we will see that the types of the two constructor values `(DtCons "List" DD 0)` and `(DtCons "List" DD 1)` are `(DtcAppType Int (DtcAppType (Datatype "List" DD) (Datatype "List" DD)))` and `(Datatype "List" DD)` respectively. In other words, the first value denotes a partially applied constructor that expects an integer and a list, while the second corresponds to a list. Note that the middle type `(Datatype "List" DD)` in the former arises from resolving the field type `(TypeRef "List")` of `listc` against `DD`.

Recall that apply values apply partially applied constructors to arguments. This uses a helper method `smt_typeof_apply_value`, which notice only applies to `SmtType.DtcAppType`. In this definition, the application is only well typed if (1) the argument type is not `SmtType.None`, and (2) matches the domain type of the constructor type. We use `Teq` (syntactic equality on SMT-LIB types) for this purpose. Note that this implies that all syntactically distinct types are considered to be distinct, and there is no notion of type equivalence.

2.5.1 Type Finiteness, Witness, Inhabitedness and Value Canonicity

We now turn our attention to defining a notion of *canonical* SMT-LIB value. This will require two prerequisite definitions. First, we will require defining a predicate for deciding whether a type is finite. Second, we require defining a "default value" for a given type, that is, a method for constructs an arbitrary value for a given type. These both will impact our definition of canonicity for map values. Our notion of inhabited types is defined alongside default values, as exactly the types for which a default value can be constructed.

```
theorem finite_type_enumerable
    (uc : Nat -> Nat) (A : SmtType)
    (hInh : smt_inhabited_type A = true)
    (hRec : smt_type_wf_rec A = true)
    (hFin : smt_is_finite_type uc A = true) :
    ∃ l : List SmtValue, ∀ v : SmtValue,
      smt_typeof_value v = A →
      smt_value_canonical uc v = true → v ∈ l

theorem fresh_typed_canonical_value_for_infinite_type
    (uc : Nat -> Nat) (A : SmtType)
    (hInh : smt_inhabited_type A = true)
    (hRec : smt_type_wf_rec A = true)
    (hInfinite : smt_is_finite_type uc A = false)
    (avoid : List SmtValue) :
    ∃ i : SmtValue,
      smt_typeof_value i = A ∧
      smt_value_canonical uc i = true ∧
      (∀ j : SmtValue, j ∈ avoid → veq j i = false)

theorem unit_type_values_default
    (uc : Nat -> Nat) (A : SmtType)
    (hInh : smt_inhabited_type A = true)
    (hRec : smt_type_wf_rec A = true)
    (hUnit : smt_is_unit_type uc A = true) :
    ∀ v : SmtValue,
      smt_typeof_value v = A →
      smt_value_canonical uc v = true →
      v = smt_type_default A

theorem nonunit_typed_canonical_nondefault_value
    (uc : Nat -> Nat) (A : SmtType)
    (hInh : smt_inhabited_type A = true)
    (hRec : smt_type_wf_rec A = true)
    (hNonUnit : smt_is_unit_type uc A = false) :
    ∃ e : SmtValue,
      smt_typeof_value e = A ∧
      smt_value_canonical uc e = true ∧
      veq e (smt_type_default A) = false
```

Figure 7: Soundness and completeness of the finite and unit type classifiers, relative to the well-formedness and inhabitedness guards. Statements 1 (`finite_type_enumerable`) and 3 (`unit_type_values_default`) are the soundness directions; statements 2 (`fresh_typed_canonical_value_for_infinite_type`) and 4 (`nonunit_typed_canonical_nondefault_value`) are the completeness directions.

First, Figure 6 defines the predicates `smt_is_finite_type` and `smt_is_unit_type`, which (for well-formed, first-class types) return true iff that type is finite, respectively *unit* (i.e. has exactly one canonical value). Both are instances of a single classifier `smt_type_bounded`, whose Boolean parameter `u` selects between the unit check (`u = true`) and the finiteness check (`u = false`). The base cases are as follows. Booleans and characters are finite but not unit; bit-vectors are finite for every bitwidth, and unit iff their bitwidth is zero. A map type is finite in one of two cases: either its element type is unit (in which case its index type is irrelevant, since all such maps store the same element everywhere), or both its index and element types are finite; it is unit iff its element type is unit. Set types are finite iff their element type is finite, and are never unit. An uninterpreted sort is finite iff the model gives it a nonzero cardinality, and unit iff that cardinality is one. Other types such as integers, reals, sequences and regular expressions are unconditionally not finite. Since uninterpreted sorts are classified using the domains chosen by the model, `smt_type_bounded` and every method that calls it, including `smt_is_finite_type` and `smt_value_canonical` below, take them as an additional argument `uc`; when a model `M` is at hand it is `M.usortDom`. We do not define finiteness for functions here, as they will not be considered first-class.

Classifying a datatype type `(Datatype s dd)` requires a fixpoint computation over its declaration block, performed by the method `smt_datatype_decl_bounded`. This method maintains an accumulator declaration block `ddB` containing the declarations classified as bounded so far, initially empty. Each pass (`smt_datatype_decl_bounded_step`) scans all declarations of `dd`: a declaration `(sF, dF)` not already in `ddB` is added to `ddB` when its definition `dF` is bounded relative to `ddB`. Here, a definition is bounded relative to `ddB` if all of its constructors have only bounded field types (and, for the unit check, if it moreover has exactly one constructor), where a field `(TypeRef s)` counts as bounded iff `s` is already in `ddB` (method `smt_field_type_bounded`), and all other field types are classified by `smt_type_bounded` as usual. Since each pass can only grow the accumulator, iterating the step once per declaration of `dd` reaches the least fixpoint. Finally, `(Datatype s dd)` is classified as bounded iff `s` occurs in the resulting accumulator.

Note that a datatype involved in a (mutually) recursive cycle is never added to the accumulator, as its references can never enter `ddB` before it does; hence recursive datatypes are classified as infinite. In practice, this is accurate for inhabited inductive datatypes (those that admit finite values). In contrast, references to non-recursive datatypes of the same declaration block are handled precisely. For example, in a block declaring an enumeration `E` and a datatype `P` whose constructor fields reference `E`, the first pass adds `E` and the second pass adds `P`, so both are correctly classified as finite. Overall, type finiteness will impact our definition of canonicity, which we describe later in this section.

The method `smt_type_default` takes a type and returns an (arbitrary) value of that type, which is guaranteed to succeed when the input type is well-formed (which we define later). Datatypes are the only non-trivial case. The key intuition is that, for inhabited datatypes, it is never productive to follow a recursive reference when constructing a default value: some constructor must be constructible without one.

In detail, for a datatype type `(Datatype s dd)`, we call `(smt_datatype_decl_default s dd dd)`, which scans the declaration block for the entry declared under name `s`. When that entry is found, say with the remaining suffix of later declarations `ddF`, we call `(smt_datatype_default s dd 0 dF ddF)`, which attempts to construct an application of one of the constructors of the definition `dF`, starting with constructor index 0. For the constructor at index `n`, the method `smt_datatype_cons_default` starts from the constructor value `(DtCons s dd n)` and applies it (via `SmtValue.Apply`) to the default values of its field types in turn, computed by `smt_field_type_default`. If computing a field default fails, the constructor is abandoned (returning `NotValue`) and we proceed to the next constructor. Hence our default values use the first constructor of the list whose fields can all be defaulted.

The key mechanism lies in how field references are treated by `smt_field_type_default`. A field `(TypeRef s')` is defaulted by `(smt_datatype_decl_default s' dd ddF)`, i.e. the referenced name is looked up only among the declarations *strictly later* in the block than the one currently being defaulted. References to the current datatype or to earlier declarations of the block thus fail (with `NotValue`), causing the enclosing constructor to be skipped; this both rules out following recursive cycles and makes the recursion terminate, as the suffix of available declarations shrinks with each reference followed. For example, for the declaration block `(cons "A" DA (cons "B" DB nil))` of the previous section, the default value of `A` follows the reference of its constructor `mkA` into `B` (which is declared later), where in turn the reference of `mkB` back to `A` fails, so the base constructor `leaf` is selected, yielding `(Apply (DtCons "A" DD 0) (DtCons "B" DD 1))`. For self references such as the `listc` constructor of `List`, the failing reference simply forces the choice of a non-recursive constructor (`listn`). This treatment of field references imposes a requirement on the *order* of a declaration block that is stronger than well-foundedness. Say that a block is in *productive order* when every datatype it declares has a constructor whose type references all name datatypes declared strictly later in the block. Only blocks in productive order have inhabited datatypes by the definitions above, whereas the notion we intend to capture — that each datatype of the block has a finite value — does not depend on the order in which the block was written. The two notions coincide only up to reordering. For a block whose recursion goes exclusively through direct constructor fields, which is the only form of recursion these definitions

```
def smt_field_type_default (dd : SmtDatatypeDecl) : SmtType -> SmtDatatypeDecl -> SmtValue
  | (SmtType.TypeRef s), ddF => (smt_datatype_decl_default s dd ddF)
  | T, ddF => (smt_type_default T)

def smt_datatype_cons_default (v : SmtValue) (dd : SmtDatatypeDecl) : SmtDatatypeCons -> SmtDatatypeDecl -> SmtValue
  | SmtDatatypeCons.unit, ddF => v
  | (SmtDatatypeCons.cons T c), ddF =>
    let _v0 := (smt_field_type_default dd T ddF)
    (if (veq _v0 SmtValue.NotValue) then SmtValue.NotValue else (smt_datatype_cons_default (SmtValue.Apply v _v0) dd c ddF))

def smt_datatype_default (s : String) (dd : SmtDatatypeDecl) (n : Nat) : SmtDatatype -> SmtDatatypeDecl -> SmtValue
  | (SmtDatatype.sum cF dF), ddF =>
    let _v0 := (smt_datatype_cons_default (SmtValue.DtCons s dd n) dd cF ddF)
    (if !(veq _v0 SmtValue.NotValue) then _v0 else (smt_datatype_default s dd (Nat.succ n) dF ddF))
  | dF, ddF => SmtValue.NotValue

def smt_datatype_decl_default (s : String) (dd : SmtDatatypeDecl) : SmtDatatypeDecl -> SmtValue
  | (SmtDatatypeDecl.cons sF dF ddF) => (if (s = sF) then (smt_datatype_default s dd 0 dF ddF) else (smt_datatype_decl_default s dd
     ddF))
  | ddF => SmtValue.NotValue

def smt_type_default : SmtType -> SmtValue
  | (SmtType.Datatype s dd) => (smt_datatype_decl_default s dd dd)
  | SmtType.Bool => (SmtValue.Boolean false)
  | SmtType.Int => (SmtValue.Numeral 0)
  | (SmtType.Map T U) =>
    let _v0 := (smt_type_default U)
    (if (veq _v0 SmtValue.NotValue) then SmtValue.NotValue else (SmtValue.Map (SmtMap.default T _v0)))
  | (SmtType.Set T) => (SmtValue.Set (SmtMap.default T (SmtValue.Boolean false)))
  | (SmtType.Seq T) => (SmtValue.Seq (SmtSeq.empty T))
  | (SmtType.FunType T U) => (SmtValue.Fun 0 T U)
  | (SmtType.USort i) => (SmtValue.UValue i 0)
  | ... => ...
  | T => SmtValue.NotValue

def smt_inhabited_type : SmtType -> Bool
  | SmtType.None => false
  | (SmtType.TypeRef _) => false
  | (SmtType.DtcAppType _ _) => false
  | (SmtType.Map _ U) => (smt_inhabited_type U)
  | (SmtType.Datatype s dd) => (smt_decl_inhabited s dd)
  | _ => true

def smt_decl_inhabited (s : String) : SmtDatatypeDecl -> Bool
  | (SmtDatatypeDecl.cons sF dF ddF) => (if (s = sF) then (smt_dt_inhabited dF ddF) else (smt_decl_inhabited s ddF))
  | SmtDatatypeDecl.nil => false

def smt_dt_inhabited : SmtDatatype -> SmtDatatypeDecl -> Bool
  | (SmtDatatype.sum c d), ddF => ((smt_dt_cons_inhabited c ddF) || (smt_dt_inhabited d ddF))
  | SmtDatatype.null, _ => false

def smt_dt_cons_inhabited : SmtDatatypeCons -> SmtDatatypeDecl -> Bool
  | (SmtDatatypeCons.cons (SmtType.TypeRef s) c), ddF => ((smt_decl_inhabited s ddF) && (smt_dt_cons_inhabited c ddF))
  | (SmtDatatypeCons.cons T c), ddF => ((smt_inhabited_type T) && (smt_dt_cons_inhabited c ddF))
  | SmtDatatypeCons.unit, _ => true
```

Figure 8: Finite default value computation and type inhabitedness.

```
theorem native_inhabited_type_iff (T : SmtType) :
    smt_inhabited_type T = true ↔ smt_type_default T ≠ SmtValue.NotValue

theorem type_default_typed (T : SmtType)
    (hInh : smt_inhabited_type T = true)
    (hRec : smt_type_wf_rec T = true) :
    smt_typeof_value (smt_type_default T) = T
```

Figure 9: The connection between inhabitedness and default values, proven in `Proofs/StaticTyping.lean`.

admit (see Section 2.6), every datatype has a finite value iff *some* permutation of the block is in productive order: sorting the datatypes by decreasing height of their smallest value gives one, since the constructor witnessing a smallest value of height `h` has fields whose smallest values have height below `h`, placing them later in that order. Assuming a productive order is therefore without loss of generality.

The definitions do not perform that reordering, however. It is discharged upstream of them instead, and only for one of the two front ends: the parser of the s-expression front end sorts a declaration block into a productive order before building it (`productiveOrder` in `Logos/Parser.lean`), which is sound but unverified, being part of the front end rather than of the checker. A block that reaches these definitions in another order is rejected, and it is rejected *as a whole*. Since `smt_decl_wf_rec` (Section 2.6) requires `smt_inhabited_type` of every datatype of the block, a single datatype whose references all point earlier makes every type of the block ill-formed, including those datatypes that do have a default value. For example, of the two SMT-LIB declarations

```
(declare-datatypes ((Tree 0) (TreeList 0))
  (((node (children TreeList))) ((tlistn) (tlistc (head Tree) (tail TreeList)))))

(declare-datatypes ((TreeList 0) (Tree 0))
  (((tlistn) (tlistc (head Tree) (tail TreeList))) ((node (children TreeList)))))
```

which declare the same two datatypes, the first is accepted and the second is rejected, and in the second neither `Tree` nor `TreeList` is a well-formed type. A proof file writing the second is accepted nonetheless, since the parser rewrites it into the first; a Lean-native proof script naming the second directly is not. Note this limitation is unrelated to the exclusion of nested recursion described in Section 2.6: a datatype such as `(declare-datatypes ((T 0)) (((mk (s (Seq T))))))` likewise has no well-formed type here, but for a different reason, and no reordering admits it.

Note also that the specification is not uniform in this respect. Finiteness of a datatype is decided by the fixpoint computation `smt_datatype_decl_bounded` described above, which does not depend on the order of the block, whereas inhabitedness is decided by the suffix walk shared by `smt_type_default` and `smt_inhabited_type`, which does. The two therefore disagree: for the block above written with `TreeList` first, `smt_is_finite_type` of a datatype of the block is unchanged by the reordering while `smt_inhabited_type` of it becomes false. Reformulating `smt_type_default` and `smt_inhabited_type` as fixpoints of the same shape, so that a reference resolves against the declarations already known to have a default value rather than against the suffix, would remove the ordering requirement; the cost is that the recursion is then no longer structural in the block, and so needs a termination argument of the kind `smt_datatype_decl_bounded` already carries.

The other definitions of default values are trivial. Maps and sets use a default (constant) map, where maps store the default value of the element type and sets store false. Sequence default value is the empty sequence. Function types have a default value which uses the reserved default identifier 0. Uninterpreted sort values use the identifier 0 as well, which every uninterpreted sort's domain contains.

In the figure, we also give a definition of inhabitedness for types via the method `smt_inhabited_type`. We define type inhabitedness structurally, following the shape of `smt_type_default` without constructing a value. Base types are inhabited, and the types `None`, `TypeRef` and `DtcAppType` are not. A map type is inhabited iff its element type is, mirroring the corresponding guard of `smt_type_default`. For a datatype type `(Datatype s dd)`, the method `smt_decl_inhabited` scans the declaration block for the entry declared under name `s`, exactly as `smt_datatype_decl_default` does, and `smt_dt_inhabited` requires *some* constructor of that entry to be inhabited. A constructor is inhabited (`smt_dt_cons_inhabited`) iff all of its fields are, where a field `(TypeRef s')` is resolved among the declarations strictly later in the block, as in `smt_field_type_default`.

Figure 9 states the connection between the two definitions. First, `native_inhabited_type_iff` says that `smt_inhabited_type T` holds exactly when `smt_type_default` succeeds in constructing a value for `T`, i.e. returns a value other than `NotValue`. This holds for every type, including types whose declaration blocks are malformed, since both methods make the same choices: they search the block for the same entry, try its constructors in the same order, and resolve each field reference against the same suffix of the block. Second, `type_default_typed` says that for an inhabited type that is moreover recursively well-formed (method `smt_type_wf_rec`, Section 2.6), the constructed

```
def smt_map_get_default : SmtMap -> SmtValue
  | (SmtMap.cons j e m) => (smt_map_get_default m)
  | (SmtMap.default T e) => e

def smt_map_entries_ordered_after (i : SmtValue) : SmtMap -> Bool
  | (SmtMap.cons j e m) => (vcmp j i)
  | m => true

def smt_map_canonical (uc : Nat -> Nat) : SmtMap -> Bool
  | (SmtMap.default T e) =>
    (!(smt_is_finite_type uc T) || (veq e (smt_type_default (smt_typeof_value e)))) &&
    (smt_value_canonical uc e)
  | (SmtMap.cons i e m) =>
      smt_value_canonical uc i &&
      smt_value_canonical uc e &&
      smt_map_canonical uc m &&
      smt_map_entries_ordered_after i m &&
      !(veq e (smt_map_get_default m))

def smt_seq_canonical (uc : Nat -> Nat) : SmtSeq -> Bool
  | (SmtSeq.empty T) => true
  | (SmtSeq.cons v s) => (smt_value_canonical uc v) && (smt_seq_canonical uc s)

def smt_value_canonical (uc : Nat -> Nat) : SmtValue -> Bool
  | (SmtValue.Binary w n) => (if (0 <= w) then (n = n % (2 ^ w)) else true)
  | (SmtValue.Char c) => (c < 196608)
  | (SmtValue.Map m) => (smt_map_canonical uc m)
  | (SmtValue.Set m) => (smt_map_canonical uc m) && (veq (smt_map_get_default m) (SmtValue.Boolean false))
  | (SmtValue.Seq s) => (smt_seq_canonical uc s)
  | (SmtValue.UValue i r) => decide (r ∈ uc i)
  | (SmtValue.RegLan r) => (re_canonical r)
  | (SmtValue.Apply f v) => (smt_value_canonical uc f) && (smt_value_canonical uc v)
  | v => true
```

Figure 10: Canonicality checks for maps, sequences, and values.

value has type `T`, where recall that the type of values can be computed by the method `smt_typeof_value`. The well-formedness hypothesis is necessary. The type of a constructor value is computed by looking up its datatype in the *whole* declaration block (`smt_dd_lookup`), whereas a field reference is defaulted from the entry found in a *suffix* of the block. The two agree when the names of the block are distinct, which `smt_decl_wf_rec` requires; for a block declaring the same name twice, the default can be constructed from one entry and typed against the other. Together the two theorems give that every well-formed type has a default value of that type, which is how inhabitedness is used throughout the remainder of this document.

Note that any datatype that is not well-founded (i.e. does not have finite values) is not inhabited by this definition. The converse does not hold: as discussed above, `smt_type_default` also fails, and hence `smt_inhabited_type` is false, on a well-founded datatype whose declaration block is not in productive order. It is important to note that the definition of `smt_inhabited_type` could alternatively have been defined by a native Lean binder, i.e. a type is inhabited iff there exists a value having that type. However, this definition does not quite coincide with our desired notion, as it admits other types (e.g. datatypes with nested recursion) to be considered inhabited. Instead, `smt_inhabited_type` captures exactly the types for which `smt_type_default` constructs a value.

Defining `smt_inhabited_type` without reference to values has a practical purpose. Inhabitedness is part of type well-formedness, which the proof checker computes when checking side conditions. A structural definition lets the checker decide it without depending on the value datatype `SmtValue` or the model semantics, so that these can use mathematical libraries (e.g. for real numbers) that the checker is never built against.

Given definitions of finiteness of types and a default value method, we are now ready to define canonicity of types. This is given by the method `smt_value_canonical` in Figure 10. If this method returns true for two syntactically distinct values having a *first-class* type, then it will be the case that those two values are semantically distinct in all models. Moreover, it will be the case that our types will be interpreted as the subset of values of their type for which `smt_value_canonical` returns true.

Note that `smt_value_canonical` by default considers values to be canonical (its default value is true). Hence, the list of cases can be seen as exceptions to when a value is not canonical.

When introducing SMT-LIB values, we remarked that there were many cases where two syntactically distinct map values denoted the same map. Our notion of canonicity rectifies this by only accepting one such array. In particular, for map values, we use the method `smt_map_canonical`. This is defined recursively. For each index `i` and element `e` pair stored in the list, we require the following things. First, `i` and `e` must themselves be canonical values. Second, the tail of the map must also be canonical. Third, the indices must be sorted. We use the strict

total ordering `vcmp` for this. In particular, the submethod `smt_map_entries_ordered_after` ensures that the index is strictly less than the next stored index, which by transitivity ensures that the index list is sorted. This criteria prevents a list of stores being equivalent, both by commuting distinct indices and by disallowing repeated equivalent indices. Finally, we must ensure that the element does not coincide with the default element stored in the map (obtainable by `smt_map_get_default`), which would also make it redundant.

The default value of a map has the following requirement: First, the default value must itself be canonical. Second, if the index type of the map is finite, then the default element stored must be the one obtained by the method `smt_type_default` in the previous section. To understand the second restriction, consider the following equivalent maps have index and element type as `Bool`:

```
m1 :=
  (SmtMap.cons (Boolean true) (Boolean true)
  (SmtMap.cons (Boolean false) (Boolean true)
    (SmtMap.default Bool (Boolean false))))
m2 := (SmtMap.default Bool (Boolean true))
```

In other words, `m1` is a map that stores `(Boolean true)` at both the index `(Boolean true)` and `(Boolean false)`, and stores `(Boolean false)` otherwise, and `m2` is a map that stores `(Boolean true)` everywhere. It is easy to see that these two maps are semantically the same, as they store `(Boolean true)` everywhere. The key thing to notice is that for finite types, it is possible to exhaust the entire domain with stores, thereby making the default value irrelevant. Without the restriction on default values, both `m1` and `m2` could be considered canonical.

For infinite types, it is impossible to exhaust the space of all possible indices. Moreover, it would be limiting to insist that a consistent default is used for a map. For instance say we have a map from `Int` to `Int`. If we insisted maps always had the same default given any index type, it would be impossible to represent a map mapping all integers to both 0 and 1, say. On the other hand, with finite types, it is possible to store values for all indices, so this restriction comes with no loss of generality for maps with finite index type. In fact, `m1` above is the canonical value of a map from `Bool` to `Bool` that stores true everywhere.

In other words, our definition of canonicty ensures that default values *never* matter for maps of finite index types (since in this case they must have a common default), and default values *always* matter for maps of infinite index types (since they cannot be fully covered by a finite set of stores).

Sets are similar to maps, but for canonical values, their default value is false, regardless of the index type. Defining canonicity for the other types is trivial. Bit-vector and character values are canonical exactly when they satisfy the same range constraints imposed by their typing: the payload of a bit-vector must be reduced modulo 2^w, and a character must lie in the legal character range. Regular language values are canonical when the base elements of the regular expression are valid character values, as checked by `re_canonical` (not shown); note this is a structural condition and does not provide canonicity in the semantic sense above, which is why equality on regular languages is defined extensionally. For the remaining composite SMT-LIB values, we simply traverse their structure and confirm that all values they are built from are also canonical.

2.6 Well-Formedness of Types

Figure 11 provides a central definition which we will use when defining the properties of SMT-LIB models we wish to consider. The key method defined in Figure 11 is `smt_type_wf`. This predicate will define exactly the set of types that are relevant to in our SMT-LIB representation.

Let us now look in detail at how `smt_type_wf` is defined. First, note that `smt_type_wf` relies in several places on a helper `smt_type_wf_component`, which returns true for types that are well-formed and can be used as component types to build other well-formed types. We also will say that a type `T` is *first-class* iff `(smt_type_wf_component T)` returns true. Note this method requires that the given type `T` is inhabited and is well-formed (based on the recursive method `smt_type_wf_rec`).

First, regular languages and function types will not be considered first-class types. They will however be permitted as top-level types. Thus, we provide them as special cases in `smt_type_wf`, where regular languages return true. Function types are considered well-formed if their argument types are first-class and their return types are well-formed. In particular, note that the return type of a function type is checked recursively by `smt_type_wf` itself rather than by `smt_type_wf_component`; this is what permits curried multi-argument function types such as `(FunType Int (FunType Int Bool))`. This definition excludes functions taking regular expressions as arguments, as well as higher-order functions (functions taking functions as arguments), since `smt_type_wf_component` returns false for both regular languages and function types.

The main recursive method for computing well-formedness of types is `smt_type_wf_rec`. At a high level, the method ensures that non-first-class types such as regular expressions, functions, and partially applied datatype

```
def smt_dt_cons_wf_rec (dd : SmtDatatypeDecl) : SmtDatatypeCons -> Bool
  | (SmtDatatypeCons.cons (SmtType.TypeRef s) c) => ((smt_dd_has_dt s dd) && (smt_dt_cons_wf_rec dd c))
  | (SmtDatatypeCons.cons T c) => ((smt_inhabited_type T) && (smt_type_wf_rec T) && (smt_dt_cons_wf_rec dd c))
  | SmtDatatypeCons.unit => true

def smt_dt_wf_rec (dd : SmtDatatypeDecl) : SmtDatatype -> Bool
  | (SmtDatatype.sum cF dF) => ((smt_dt_cons_wf_rec dd cF) && (smt_dt_wf_rec dd dF))
  | SmtDatatype.null => true

def smt_decl_wf_rec (dd : SmtDatatypeDecl) : SmtDatatypeDecl -> Bool
  | (SmtDatatypeDecl.cons s d ddF) =>
      ((smt_dt_wf_rec dd d) && (smt_inhabited_type (SmtType.Datatype s dd)) && (smt_decl_wf_rec dd ddF) && !(smt_dd_has_dt s ddF))
  | SmtDatatypeDecl.nil => true

def smt_type_wf_rec : SmtType -> Bool
  | (SmtType.Datatype s dd) => ((smt_dd_has_dt s dd) && (smt_decl_wf_rec dd dd))
  | (SmtType.TypeRef s) => false
  | (SmtType.Seq x1) => ((smt_inhabited_type x1) && (smt_type_wf_rec x1))
  | (SmtType.Map x1 x2) =>
      ((smt_inhabited_type x1) && (smt_type_wf_rec x1) && (smt_inhabited_type x2) && (smt_type_wf_rec x2))
  | (SmtType.Set x1) => ((smt_inhabited_type x1) && (smt_type_wf_rec x1))
  | (SmtType.FunType x1 x2) => false
  | (SmtType.DtcAppType x1 x2) => false
  | SmtType.None => false
  | SmtType.RegLan => false
  | U => true

def smt_type_wf_component (T : SmtType) : Bool :=
  (smt_inhabited_type T) && (smt_type_wf_rec T)

def smt_type_wf : SmtType -> Bool
  | SmtType.RegLan => true
  | (SmtType.FunType T U) => (smt_type_wf_component T) && (smt_type_wf U)
  | T => (smt_type_wf_component T)
```

Figure 11: Type well-formedness.

constructors do not occur as component types. It also ensures that *every* field of every constructor of a datatype is inhabited and well-formed, whereas note that `smt_inhabited_type` only requires that *some* constructor is inhabited.

For a datatype type `(Datatype s dd)`, well-formedness requires that the name `s` is declared in `dd` (via `smt_dd_has_dt`) and that the declaration block itself is well-formed, checked by the helper `smt_decl_wf_rec`. The latter visits each declaration of the block in turn and requires, for the declaration of a datatype `d` under name `s'`: (1) the constructors of `d` are well-formed (via `smt_dt_wf_rec` and `smt_dt_cons_wf_rec`), (2) the datatype `(Datatype s' dd)` is inhabited, and (3) the name `s'` is not declared again in the remainder of the block (`!(smt_dd_has_dt s' ddF)`), which rules out the duplicate declarations (aliasing) discussed in Section 2.2. A constructor is well-formed if for each of its field types `T`, either `T` is a type reference whose name is declared in the block (i.e. in scope), or otherwise `T` is an inhabited type that is well-formed, which we check recursively. Note that this definition directly accounts for mutually recursive datatypes, as all datatypes of the group are declared in the same block against which type references are checked.

Note that type references are only permitted as direct fields of constructors. A bare type reference is not well-formed, given the case for `(SmtType.TypeRef s)` in `smt_type_wf_rec`, which returns false.

The next cases in `smt_type_wf_rec` cover composite types like maps, sequences and sets. Since their component types (e.g. element and index types of maps, or element types of sequences) are checked by `smt_type_wf_rec` itself, a type reference occurring beneath such a type constructor is not well-formed. This disallows nested recursive datatypes. It is also important to note that composite types built from uninhabited types are themselves considered not well-formed. This prevents us from otherwise having e.g. sets of an uninhabited type be inhabited via the empty set; sequences are guarded for the same reason.

The remaining cases in `smt_type_wf_rec` are exceptions for other types that we disallow as first-class types. As mentioned, we do not have regular expressions or function types as first class. We also disallow partially applied datatype constructors as a first-class type. Finally, we disallow the error type `None`. All other types (e.g. the atomic base types) are first-class.

To understand the intuition for the precise motivation for the definition of `smt_type_wf`, it is important to recall our definition of canonicity of values. Assume we have two canonical values `v1` and `v2` of first-class type `T`, that is, `smt_typeof_value`$v1 = $ `smt_typeof_value`$v2 = T$, and `smt_value_canonical`$v1 = $ `smt_value_canonical`$v2 =$ true. In this case, we say that `smt_typeof_value`$v1$ and `smt_typeof_value`$v2$ are semantically equivalent iff they are syntactically equivalent. Concretely, our definition of SMT-LIB model evaluation `smt_model_eval` for equality

```
def model_lookup (M : SmtModel) (isVar : Bool) (s : String) (T : SmtType) : SmtValue :=
  M.values { isVar := isVar, name := s, ty := T }

def model_fun_lookup (M : SmtModel) (fid : ModelId) (T U : SmtType) : SmtValue -> SmtValue :=
  M.nativeFuns fid T U

def eval_fun_apply (M : SmtModel) (fid : ModelId) (T U : SmtType) (i : SmtValue) : SmtValue :=
  if (fid = 0) then (smt_type_default U) else (model_fun_lookup M fid T U i)

def usort_dom_wf (uc : Nat -> Set ModelId) : Prop :=
  ∀ i, (0 : ModelId) ∈ uc i ∧
    ((uc i).Infinite -> ∀ n : Nat, ∃ r ∈ uc i, (n : ModelId) ≤ |r|)

def model_wf (M : SmtModel) : Prop :=
  (∀ isVar s T, smt_type_wf T = true ->
    smt_typeof_value (model_lookup M isVar s T) = T ∧
    smt_value_canonical M.usortDom (model_lookup M isVar s T) = true) ∧
  (∀ fid A B i,
    smt_type_wf (SmtType.FunType A B) = true ->
    smt_typeof_value i = A ->
    smt_typeof_value (eval_fun_apply M fid A B i) = B ∧
      smt_value_canonical M.usortDom (eval_fun_apply M fid A B i) = true) ∧
  usort_dom_wf M.usortDom
```

Figure 12: Total typed model conditions.

(`eq`) will use syntactic comparison over values (`veq`) for all first-class types, and will contain a special case for regular expressions.

2.7 Well-Formed Models

We now have the required definitions to define well-formed models, as given in Figure 12. In particular, we define `model_wf` to be a property of models that holds when its value and function fields are well-formed.

In detail, the first clause says that for all keys (which consist of a flag `isVar` denoting whether we are looking up a variable, a string identifier `s`, and a type `T`), if `T` is a well-formed type (based on the definition from Figure 11), then the map `M.values` stores a value `v` that has the given type (i.e. `T`), and moreover that value is *canonical*. It will be the case that our model evaluator only returns canonical values, the latter requirement can be seen as a base case on the leafs when interpreting a term (i.e. its variables and free constants).

The second clause says that for all identifiers and types, the `M.nativeFuns` has similar requirements, but where now the functions stored in the domain behave like properly typed functions of the given type. Function behavior is accessed through the method `eval_fun_apply`, which behaves as the direct lookup `model_fun_lookup` except on the reserved default function identifier 0, for which it returns the default value of the range type regardless of the model; this guarantees that the default function value from Figure 8 is well-behaved in every model. Model evaluation of function applications will go through the same method. In particular, the second clause requires that when `(SmtType.FunType A B)` is well-formed, for all inputs `i` whose type is `A`, applying `(eval_fun_apply M fid A B)` to `i` results in a value of type `B`, and moreover the returned value is canonical.

The third clause restricts the domains of uninterpreted sorts, as described in Section 2.4.

Notice that for the first two clauses, we insist that the respective components of `M` are total mappings.

2.8 Terms

We now turn our attention to SMT-LIB terms. Figure 13 gives a representative sample of our definitions for SMT-LIB terms.

As before, we use an error constructor `None` to denote an error value for terms. We then provide a list of standard atomic terms, which carry native Lean datatypes as arguments.

The next group of operators shows our encoding of ordinary SMT-LIB operators. We use a sanitization pass on SMT-LIB identifiers, e.g. `eq` denotes SMT-LIB equality `=`; other constructors (e.g. `not`, `and`) coincide with their SMT-LIB name. It is important to note that for these SMT-LIB operators, we use a first-order representation where the arguments of these operators are arguments to these constructors. Effectively, this hardcodes the application of interpreted functions in SMT-LIB to be fully applied. Note that SMT-LIB does not contain any variadic operators, so this is not a limitation.[5] This design decision was made since we are not considering higher-order extensions of

[5]It does however support syntax for providing arbitrary inputs to many associative operators such as `and`, which is equivalent to chaining together binary applications of that operator.

```
inductive SmtTerm : Type where
  | None : SmtTerm
  | Boolean : Bool -> SmtTerm
  | Numeral : Int -> SmtTerm
  | Rational : Rat -> SmtTerm
  | String : String -> SmtTerm
  | Binary : Int -> Int -> SmtTerm

  | eq : SmtTerm -> SmtTerm -> SmtTerm
  | not : SmtTerm -> SmtTerm
  | and : SmtTerm -> SmtTerm -> SmtTerm
  | select : SmtTerm -> SmtTerm -> SmtTerm
  | store : SmtTerm -> SmtTerm -> SmtTerm -> SmtTerm
  ...

  | Apply : SmtTerm -> SmtTerm -> SmtTerm
  | DtCons : String -> SmtDatatypeDecl -> Nat -> SmtTerm
  | DtSel : String -> SmtDatatypeDecl -> Nat -> Nat -> SmtTerm
  | DtTester : String -> SmtDatatypeDecl -> Nat -> SmtTerm
  | exists : String -> SmtType -> SmtTerm -> SmtTerm
  | forall : String -> SmtType -> SmtTerm -> SmtTerm
  | Var : String -> SmtType -> SmtTerm
  | UConst : String -> SmtType -> SmtTerm
deriving Repr, DecidableEq, Inhabited
```

Figure 13: A representative template of SMT term constructors.

SMT-LIB, and moreover higher-order treatments of SMT-LIB often avoid reasoning about interpreted functions as first class objects (they can equivalently be represented as lambda functions). Overall, this representation limits the expressiveness of our embedding but greatly simplifies several dimensions of reasoning in our proofs.

The next constructor `Apply` is used for two purposes: for applying uninterpreted functions, and for applying datatype constructors. In contrast to interpreted functions, these functions can have arbitrary (fixed) arity based on the user's definition. Thus, these are treated by curried applications.

The next two terms denote datatype constructors and datatype selectors. Datatype constructors mirror their structure from SMT-LIB values: they carry the name of their datatype, its declaration block, and the index of the constructor. Datatype selectors take the same name and declaration block, followed by two natural numbers: the index of the constructor, and the index of the field within that constructor. For instance, the head and tail selectors from the example declaration `DD` for `List` from the previous section are denoted as `(DtSel "List" DD 0 0)` and `(DtSel "List" DD 0 1)`. Note that these indices may be out of bounds, in which case the term will be ill-typed. The next constructor `DtTester` has the same arguments as `DtCons`, denoting the SMT-LIB discriminator (tester) for the constructor with the given index. Beyond the constructors shown in Figure 13, the embedding contains further binder-like constructors, notably `choice` (Hilbert choice, discussed below) and `bind` (a let-style binder used only internally by the translation from Eunoia terms), as well as the internal operators `map_diff` and `seq_diff` used to witness disequalities of arrays, sets and sequences.

The next two constructors (`forall` and `exists`) denote the SMT-LIB standard universal and existential binders. For each, the first two arguments are a string and a type, which together denote an identifier for a variable. The third argument denotes the body of the quantified formula, which is expected to be of Boolean type. Variables (`Var`) within that body are considered to be bound by the (innermost) instance of a binder that shares their identifier. For example, the SMT-LIB formula `(forall ((x Int)) (= x 0))` is represented in this encoding as:

```
(forall "x" Int (eq (Var "x" Int) (Numeral 0)))
```

This representation has some of the same complications as our representation of datatypes, which we describe briefly here. First, a formula may have free variables, i.e. variables that are not bound by a binder, e.g.:

```
(eq (Var "x" Int) (Numeral 0))
```

Second, a formula may contain aliased variables, e.g.:

```
(forall "x" Int (exists "x" Int (eq (Var "x" Int) (Numeral 0))))
```

Both formulas will be considered well-formed in our definition. For the former, the semantics of a formula with free variables follows naturally from the definition of a SMT-LIB model, namely a variable has a fixed interpretation in a model. For the latter, our semantics of quantified formulas will be bottom-up, where inner quantified formulas essentially override the interpretation of variables, so that `(Var "x" Int)` is effectively bound by the innermost binder, in this case, it is extentionally bound. Note that we do not include a constructor for `lambda`, as our

```
def smt_typeof_guard_wf (T : SmtType) (U : SmtType) : SmtType :=
  (if (smt_type_wf T) then U else SmtType.None)

def smt_typeof_not : SmtType -> SmtType
  | SmtType.Bool => SmtType.Bool
  | T => SmtType.None

def smt_typeof_eq (T : SmtType) (U : SmtType) : SmtType :=
  (smt_typeof_guard T (if (Teq T U) then SmtType.Bool else SmtType.None))

def smt_typeof_arith_overload_op_2 : SmtType -> SmtType -> SmtType
  | SmtType.Int, SmtType.Int => SmtType.Int
  | SmtType.Real, SmtType.Real => SmtType.Real
  | T, U => SmtType.None

def smt_typeof_select : SmtType -> SmtType -> SmtType
  | (SmtType.Map T U), V => (if (Teq T V) then U else SmtType.None)
  | T, U => SmtType.None

def smt_typeof_dt_cons_rec (T : SmtType) : SmtDatatype -> Nat -> SmtType
  | (SmtDatatype.sum SmtDatatypeCons.unit d), Nat.zero => T
  | (SmtDatatype.sum (SmtDatatypeCons.cons U c) d), Nat.zero =>
     (SmtType.DtcAppType U (smt_typeof_dt_cons_rec T (SmtDatatype.sum c d) Nat.zero))
  | (SmtDatatype.sum c d), (Nat.succ n) => (smt_typeof_dt_cons_rec T d n)
  | d, n => SmtType.None

def smt_typeof_apply : SmtType -> SmtType -> SmtType
  | (SmtType.FunType T U), V => (smt_typeof_guard T (if (Teq T V) then U else SmtType.None))
  | (SmtType.DtcAppType T U), V => (smt_typeof_guard T (if (Teq T V) then U else SmtType.None))
  | T, U => SmtType.None

def smt_typeof : SmtTerm -> SmtType
  | (SmtTerm.Boolean b) => SmtType.Bool
  | (SmtTerm.Numeral n) => SmtType.Int
  | (SmtTerm.Binary w n) =>
     if 0 <= w ∧ n = n % (2 ^ w) then (SmtType.BitVec (Int.toNat w)) else SmtType.None
  | (SmtTerm.not x1) => (smt_typeof_not (smt_typeof x1))
  | (SmtTerm.eq x1 x2) => (smt_typeof_eq (smt_typeof x1) (smt_typeof x2))
  | (SmtTerm.exists s T x1) =>
     if smt_typeof x1 = SmtType.Bool then (smt_typeof_guard_wf T SmtType.Bool) else SmtType.None
  | (SmtTerm.plus x1 x2) => (smt_typeof_arith_overload_op_2 (smt_typeof x1) (smt_typeof x2))
  | (SmtTerm.select x1 x2) => (smt_typeof_select (smt_typeof x1) (smt_typeof x2))
  | (SmtTerm.Apply f x1) => (smt_typeof_apply (smt_typeof f) (smt_typeof x1))
  | (SmtTerm.DtCons s dd i) =>
     let _v0 := (SmtType.Datatype s dd)
     (smt_typeof_guard_wf _v0 (smt_typeof_dt_cons_rec _v0 (smt_dt_resolve (smt_dd_lookup s dd) dd) i))
  | ... => ...
```

Figure 14: Types for SMT terms.

formulation does not consider higher-order constructs. CPC will define one further binder for a variant of Hilbert's choice operator. This operator however is specific to the implementation of the CPC calculus, so we omit it from discussion here.

The last term constructor `UConst` denotes free (uninterpreted) constants. Note that a free constant may have a function type. These have the same structure as variables, and will have a model semantics that mirrors variables, where recall that an SMT-LIB model contains a parallel mapping for variables and constants. In contrast, the interpretation of variables is updatable in model (e.g. when evaluating binders), where the interpretation of a constant is fixed.

2.9 Types of SMT-LIB Terms

Figure 14 gives our definition for the types of SMT-LIB terms. At a high-level, this is a structurally recursive method that first computes the types the arguments of the term and based on the argument types constructs a return type. We give a representative set of cases in the definition of `smt_typeof`.

First, atomic interpreted constants are trivial, e.g. Boolean constants are Bool, integer constants are Int, etc. Bit-vector constants (`Binary`) are well-typed iff they have a non-negative bitwidth `w` and a value `n` within the range $[0, 2^w)$.

For the remaining operators, it is important to note that, since our definition of SMT-LIB terms do not allow us to specify partially applied interpreted functions, the type rules we give are for fully applied terms. We give some of the cases above, for example, for SMT-LIB `not`, the method `smt_typeof_not` returns the Boolean type iff the argument is Boolean. For SMT-LIB equality `eq`, the method for computing its type `smt_typeof_eq` takes two types

```
def smt_model_eval_not : SmtValue -> SmtValue
  | (SmtValue.Boolean b) => (SmtValue.Boolean !b)
  | v1 => SmtValue.NotValue

def str_in_re : String -> SmtRegLan -> Bool
  ...

open Classical in
noncomputable def re_ext_eq (r1 r2 : SmtRegLan) : Bool :=
  if (∀ s : String, string_valid s -> (str_in_re s r1 = str_in_re s r2))
    then true else false

def smt_model_eval_eq : SmtValue -> SmtValue -> SmtValue
  | (SmtValue.RegLan r1), (SmtValue.RegLan r2) => (SmtValue.Boolean (re_ext_eq r1 r2))
  | v1, v2 => (SmtValue.Boolean (veq v1 v2))

open Classical in
noncomputable def eval_texists (M : SmtModel) (s : String)
    (T : SmtType) (body : SmtTerm) : SmtValue :=
  SmtValue.Boolean (∃ v : SmtValue,
      smt_typeof_value v = T ∧
        smt_value_canonical M.usortDom v = true ∧
        smt_model_eval (model_push M s T v) body = (SmtValue.Boolean true))

def smt_model_eval_apply (M : SmtModel) : SmtValue -> SmtValue -> SmtValue
  | v, SmtValue.NotValue => SmtValue.NotValue
  | (SmtValue.DtCons s dd n), i => (SmtValue.Apply (SmtValue.DtCons s dd n) i)
  | (SmtValue.Apply f v), i => (SmtValue.Apply (SmtValue.Apply f v) i)
  | (SmtValue.Fun s T U), i => (eval_fun_apply M s T U i)
  | v, i => SmtValue.NotValue

noncomputable def smt_model_eval (M : SmtModel) : SmtTerm -> SmtValue
  | (SmtTerm.Boolean b) => (SmtValue.Boolean b)
  | (SmtTerm.Numeral b) => (SmtValue.Numeral b)
  ...
  | (SmtTerm.not x1) => (smt_model_eval_not (smt_model_eval M x1))
  | (SmtTerm.eq x1 x2) => (smt_model_eval_eq (smt_model_eval M x1) (smt_model_eval M x2))
  | (SmtTerm.exists s T x1) => (eval_texists M s T x1)
  | (SmtTerm.DtCons s dd i) => (SmtValue.DtCons s dd i)
  | (SmtTerm.Apply f x1) =>
      (smt_model_eval_apply M (smt_model_eval M f) (smt_model_eval M x1))
  | (SmtTerm.Var s T) => (model_lookup M true s T)
  | (SmtTerm.UConst s T) => (model_lookup M false s T)
  | ... => ...
```

Figure 15: Core skeleton of SMT term evaluation in a model.

`T` and `U`. If these are syntactically equal types, then the equality has type Bool, otherwise it is ill-typed. Again notice that we consider only syntactically equivalent types here, i.e. there is no notion of type equivalence. Second, note that we permit equalities between terms whose types are not well-formed (apart from if the terms are not well typed, i.e. have type `None`).

Our type for quantifiers `exists` first checks whether its third argument (its body) has Bool type, if so, then Bool is returned provided that the quantified type is well-formed (via helper `smt_typeof_guard_wf`). This prevents quantified formulas over types that are not well-formed, which would have an unintuitive semantics based on our model evaluation for `exists` as defined earlier.

The type of arithmetic operators like `plus` handle overloading Int and Real, i.e. plus is well-typed if both of its arguments are Int, or if both arguments are Real. Array select (`select`) has type given by `smt_typeof_select`, which succeeds iff the index type of map we are selector from matches the type of the argument. The generic `Apply` term can either refer to an application of an uninterpreted function or a datatype constructor. These cases are handled similarly. The type of datatype constructor terms is analogous to the typing for datatype constructor values, as given in Figure 5: the constructor list is retrieved and made self-contained via `(smt_dt_resolve (smt_dd_lookup s dd) dd)`, guarded by well-formedness of the datatype type `(Datatype s dd)`. The code additionally contains dedicated cases (elided in the figure) for applications of datatype selectors and testers: an applied selector `(DtSel s dd i j)` is typed like a function from `(Datatype s dd)` to the resolved type of field `j` of constructor `i`, and an applied tester `(DtTester s dd i)` is typed like a predicate on `(Datatype s dd)`, guarded by the constructor index being in bounds.

2.10 Model Evaluation

Figure 15 gives a sample of cases for model evaluation, which takes as input an SMT-LIB model and a term,

and returns the evaluation of that term in the model. We will prove several theorems about this method later. It is important to note that the method is marked `noncomputable` in Lean, since several cases rely on native Lean quantification. Thus, as mentioned, model evaluation cannot be used in the Logos executable, instead it is a specification that we will base our correctness proof upon. It is also important to note that the model evaluation method `smt_model_eval` is largely structurally recursive. For example, for a majority of operators, it builds values by first evaluating each direct subterm of the current one and then combining their results.

In the method `smt_model_eval`, we present a sample of the cases. The first set of cases is for the atomic interpreted constant terms, e.g. Booleans, integer numerals, etc. These simply evaluate to their corresponding SMT-LIB value. We then present the evaluator for SMT-LIB `not`. This recursively calls model evaluation on its argument which is passed to `smt_model_eval_not`. If the argument was a Boolean value, then its payload is negated. Otherwise, we return an error. A majority of evaluation for SMT-LIB operators is defined in this manner.

We also present several of the key definitions for evaluation. In particular, the next case is for SMT-LIB equality. We define the method `smt_model_eval_eq` for this, which takes the result of evaluating both sides of the equality. Recall that our definitions of canonicity implied that for most types, two canonical values are syntactically distinct iff they are semantically distinct. Moreover, we will prove as an invariant of our model evaluation that for well-formed models, only canonical values are returned. This is reflected in our definition of `smt_model_eval_eq`. For all types (apart from regular expressions), model evaluation for equality reduces to a syntactic comparison of the two values (via `veq`). For regular expression values (`SmtValue.RegLan`), we instead rely on an extensional definition of equality, which states that two regular expressions are equal if regular expression membership is equivalent for all strings. This definition depends on a method `str_in_re` (not given) for deciding whether a concrete string is a member of a (ground) regular expression. This method internally is based on derivatives of strings. Note that the quantification in `re_ext_eq` is restricted to *valid* strings (via `string_valid`), i.e. those whose characters are within the legal SMT-LIB character range.

Note that function values are treated *intensionally* in our definition of equality: the values `(Fun n U T)` and `(Fun m U T)`, i.e. two function values over the same type, are unconditionally considered disequal when `n` and `m` are distinct identifiers. This is even if the functions they refer to in the model (via `M.nativeFuns`) are identical. This is an acceptable definition, as we do not consider functions to be a first-class type in our representation.

The next case we give is for the SMT-LIB quantifier `exists`. This case involves an auxiliary definition `eval_texists`. This returns a Boolean whose value is determined by a Lean binder, which existentially quantifies over SMT-LIB values. It states that if there exists a (canonical) value of the same type as the binder such that the body evaluates to true under the model `(model_push M s T v)`. Recall that the definition of `model_push` from Figure 3 updates the variable with name `s` and type `T` to be value `v`. In other words, the recursive model evaluation is done on a modified version of the current model where the variable specified by the binder is *replaced* by `v`. Note that this definition has the advantage that we do not need to define a substitution function on terms, nor do we need have SMT-LIB terms depend on SMT-LIB values (the two datatypes do not depend on one another). The case for universal quantification `forall` (not given) is similar.

The next two cases handle datatype constructors and applications of datatype constructors and functions. Firstly, datatype constructor terms are interpreted as their corresponding datatype constructor value. For applications, we first evaluate the function and argument and call the method `smt_model_eval_apply`. This has 5 cases. First, if the argument failed to evaluate, we fail to evaluate. Second, if the head was a datatype constructor, then we construct the (partially) applied datatype constructor value via `SmtValue.Apply`. The third case handles when the head itself was a partially applied constructor. It is the case by induction that the only `SmtValue.Apply` values returned by model evaluation necessarily have datatype constructors as their heads. In the fourth case, if the head was a function value, we look up its behavior in the model and apply it to the argument, via the method `eval_fun_apply` from Figure 12. Otherwise, we fail to evaluate.

Applications of datatype selectors and testers are given dedicated cases in `smt_model_eval` (elided in Figure 15). A tester application evaluates to whether the head of the evaluated argument is the constructor value with the tester's index. A selector application, when its argument is an application of the matching constructor, evaluates to the corresponding argument of that application; when applied to a value built from a *different* constructor, its result is looked up from a distinguished function symbol in the model, reflecting that SMT-LIB leaves wrongly-applied selectors unconstrained. A similar mechanism using distinguished model symbols is used for other underspecified operations, e.g. division by zero and out-of-bounds sequence access.

The final cases of `smt_model_eval` are for variables (`Var`) and free constants (`UConst`). These are handled similarly in the model, where the model lookup takes a flag denoting whether or not the lookup is for a variable.

More advanced cases of model evaluation are given in the appendix. (TODO)

```
theorem model_wf_nonvacuous :
    ∃ M : SmtModel, model_wf M

theorem type_preservation
    (M : SmtModel)
    (hM : model_wf M)
    (t : SmtTerm)
    (ht : smt_typeof t ≠ SmtType.None) :
    smt_typeof_value (smt_model_eval M t) = smt_typeof t

theorem model_eval_canonical
    (M : SmtModel)
    (hM : model_wf M)
    (t : SmtTerm)
    (hTy : smt_typeof t ≠ SmtType.None) :
    smt_value_canonical M.usortDom (smt_model_eval M t) = true
```

Figure 16: Main type-preservation, canonicality, and model-existence statements.

2.11 Properties of Models, Model Evaluation

Figure 16 gives three central theorems about our definitions thus far. In particular, the first theorem states simply that a well-formed model exists. In other words, our definitions are not vacuous as we can show that at least one model exists (in fact, infinitely many models exist). Note that this must be stated as a theorem due to our definitions of model well-formedness, which are stated using native (universal) Lean quantifiers.

The second theorem states that given a well-formed model `M` and well-typed term `t`, model evaluation preserves type. Note that terms and values share the same representation of types (`SmtType`), hence their types can be compared directly. This method is proved by induction on the structure of terms.

The third theorem is similar and proves that given a well-formed model `M` and well-typed term `t`, the model evaluation of the term is a *canonical* value. Recall that our definition of canonical values allows us to use syntactic equality to check semantic equivalence. This theorem essentially allows us (among other things) to argue that model evaluator for equality has an appropriate definition. This theorem relies on the fact that for well-formed models, in the base case, variables and constants are mapped to canonical values. It then proceeds by induction on the structure of terms.

3 Specification

This section presents the *specification* layer of Logos. This layer introduces the terms used by the executable checker. We refer to these as *Eunoia terms*, as they are compiled based on the input Eunoia signature. These are captured in the Lean datatype `Term`. Eunoia terms represent both checker-level terms and types uniformly.

The key methods we define in this layer are to define a reduction from Eunoia term to SMT terms and types, and then to define a notion of satisfiability (`eo_satisfiability`) based on this reduction. We first will introduce the term embedding.

3.1 Eunoia Terms

Figure 17 introduces our definition of Eunoia terms (`Term`). As mentioned, rules in our proof checker can be seen as methods taking a list of Eunoia terms and returning a Eunoia term (the conclusion), which is expected to be a term of type Bool. Since we use a single unified representation, proof rules can essentially be seen as *untyped* manipulations over terms in this deep embedding. We now introduce the specifics of the term embedding.

First, like the other datatypes introduced in this document, we have an error term, denoted `Stuck`.[6] Before describing user-defined operators, the constructor `Apply` denotes higher-order application, taking a term to apply and its argument. We use a curried style for applying terms, e.g. the application of a binary function `f` to arguments `a` and `b` is denoted `(Apply (Apply f a) b)`.

The next constructors `UOp`, `UOp1`, `UOp2`, etc. denote all symbols introduced in the user-defined Eunoia signature. These each rely on enumeration types (e.g. `UserOp`) which are automatically compiled based on the Eunoia signature we are verifying. The constructors for user-defined operators are stratified based on the number of indexed arguments

[6] We call the error term `Stuck` as this reflects the described operational semantics of Eunoia, where a proof rule that fails to execute may become "stuck".

```
/- Ordinary user operators. -/
inductive UserOp : Type where
  | Int : UserOp
  | Real : UserOp
  | BitVec : UserOp
  ...
  | not : UserOp
  | eq : UserOp
  | and : UserOp
  ...

/- User operators with indexed arguments -/
inductive UserOp1 : Type where
  | repeat : UserOp1
  | zero_extend : UserOp1
  ...

inductive UserOp2 : Type where
  | extract : UserOp2
  ...

inductive Term : Type where
  | Stuck : Term
  | UOp : UserOp -> Term
  | UOp1 : UserOp1 -> Term -> Term
  | UOp2 : UserOp2 -> Term -> Term -> Term
  ...
  | Apply : Term -> Term -> Term
  | eo_List : Term
  | eo_List_nil : Term
  | eo_List_cons : Term
  | Bool : Term
  | FunType : Term
  | Type : Term
  | Boolean : Bool -> Term
  | Numeral : Int -> Term
  | Rational : Rat -> Term
  | String : String -> Term
  | Binary : Int -> Int -> Term
  | DatatypeType : String -> DatatypeDecl -> Term
  | DatatypeTypeRef : String -> Term
  | DtcAppType : Term -> Term -> Term
  | DtCons : String -> DatatypeDecl -> Nat -> Term
  | DtSel : String -> DatatypeDecl -> Nat -> Nat -> Term
  | USort : Nat -> Term
  | UConst : Nat -> Term -> Term
  | Var : Term -> Term -> Term
deriving Repr, DecidableEq, Inhabited, Ord

/-- Syntactic equality for Eunoia terms --/
def teq : Term -> Term -> Bool
  | x, y => decide (x = y)
```

Figure 17: Core Eunoia term syntax.

they have.[7]

First, ordinary operators (`UOp`) take as argument the enumeration type `UserOp`, which denotes the operator they are. This enumeration type includes all user operators that are not indexed by an argument. This includes both types (e.g. `Int`) and terms (e.g. `not`, `and`).

Indexed arguments (`UOpN`) for `N = 1, 2, ...` are terms that take as argument the enumeration type `UserOpN` denoting the operator they are, as well as `N` term arguments. These term arguments are part of the argument to the *constructor* `UOpN`. Note this means that indexed operators are thus considered "atomic" with respect to our applicative encoding of terms. For example, the SMT-LIB term `((_ extract 2 0) t)` is represented as:

```
(Apply (UOp2 UserOp2.extract (Numeral 2) (Numeral 0)) t)
```

and not:

```
(Apply (Apply (Apply (UOp UserOp.extract) (Numeral 2)) (Numeral 0)) t)
```

In other words, `(_ extract 2 0)` is intuitively an atomic term, and not an application of `extract` to two integers. This has an important impact on what is possible to infer for certain proof rules. For instance, a generic congruence rule *cannot* be used to infer that e.g. `((_ extract 2 0) t)` is equivalent to `((_ extract x 0) t)` if `x = 2`. Essentially, indexed arguments allow us to consider their indices to be opaque from outside reasoning.

The remaining operators define the builtin operators of Eunoia. First, Eunoia has a builtin notion of lists with constructors `eo_List_cons` and `eo_List_nil`. The list type is the constructor `eo_List`. The next set of operators denote the other builtin types of Eunoia including Booleans (`Bool`), the arrow type (`FunType`), and the type of types (`Type`). The next five constructors denote the SMT-LIB literal categories supported by Eunoia. Note that these constructors carry native Lean types that denote their values, similar to our definition of SMT-LIB terms and values.

The next 5 constructors are used for representing user-defined SMT-LIB datatypes. Eunoia inherits the notion of SMT-LIB datatypes. These constructors mirror the declaration-based definitions of SMT-LIB datatype types and terms, as described in the previous section. In particular, `DatatypeType` carries a name and a declaration block (here of the Eunoia-level type `DatatypeDecl`, which mirrors `SmtDatatypeDecl` except that constructor field types are Eunoia terms), `DatatypeTypeRef` mirrors the type reference `TypeRef`, and `DtcAppType` is the type of partially applied constructor terms. Datatype constructor and selector terms are given by `DtCons` and `DtSel`, carrying a name, a declaration block and their indices as in the SMT-LIB embedding. It is important to note that each of the arguments of these 5 constructors are arguments to the constructors. In other words, they are indices of the terms. This is important since the semantics of matching on Eunoia terms does *not* have access to the internals of their definitions (e.g. matching on a generic apply term in Eunoia cannot decompose the arguments of a user-defined datatype constructor). This accurately reflects the operational semantics of Eunoia.

The next constructor `USort` corresponds to user-defined uninterpreted sorts. The constructor `UConst` corresponds to user-defined constants. This takes as argument an identifier (given by a natural number) and a term (denoting the type of the constant). Notice this mirrors our definition of SMT-LIB constants (`SmtTerm.UConst`). The final constructor `Var` denotes a Eunoia variable. Variables in Eunoia are first-class objects taking their name (intended to be a string literal) and their type. Based on the operational semantics of Eunoia, these are indexed arguments, i.e. variables are considered atomic terms indexed by their name and type. Note that type constructors such as `BitVec` are *ordinary* operators applied to their arguments via `Apply` (e.g. the bit-vector type of width 8 is `(Apply (UOp UserOp.BitVec) (Numeral 8))`), while `UserOp1` contains operators such as `repeat`, `zero_extend` and the datatype tester and updater operators `is` and `update`; the stratification continues with `UserOp2`, `UserOp3`, etc. as needed by the signature.

3.2 Translation from Eunoia term to SMT terms

Figure 18 gives a representative sample of the two translation methods of the specification layer: `eo_to_smt_type`, mapping Eunoia terms that denote types to `SmtType`, and `eo_to_smt`, mapping Eunoia terms to `SmtTerm`. Both are defined by pattern matching on the shape of the Eunoia term and return the respective error element (`SmtType.None` resp. `SmtTerm.None`) when no case applies; composite cases guard against errors in their subterms (e.g. via `smt_typeof_guard`).

The datatype cases directly reflect the shared declaration-based representation. A Eunoia declaration block is translated pointwise by `eo_to_smt_datatype_decl`, which maps each declared datatype's constructor field types through `eo_to_smt_type`. A Eunoia datatype type `(DatatypeType s dd)` is translated to `(SmtType.Datatype s`

[7] Some SMT-LIB operators are indexed, e.g. bitvector extract (SMT-LIB syntax `(_ extract n m)`. Indexed arguments are denoted by the `:opaque` attribute in Eunoia signatures.

```
def eo_to_smt_datatype : Datatype -> SmtDatatype
  ...

def eo_to_smt_datatype_decl : DatatypeDecl -> SmtDatatypeDecl
  | (DatatypeDecl.cons s d dd) => (SmtDatatypeDecl.cons s (eo_to_smt_datatype d) (eo_to_smt_datatype_decl dd))
  | DatatypeDecl.nil => SmtDatatypeDecl.nil

def eo_to_smt_type : Term -> SmtType
  | Term.Bool => SmtType.Bool
  | (Term.UOp UserOp.Int) => SmtType.Int
  ...
  | (Term.Apply (Term.UOp UserOp.BitVec) (Term.Numeral n1)) =>
    (if (0 <= n1) then (SmtType.BitVec (Int.toNat n1)) else SmtType.None)
  | (Term.Apply (Term.Apply Term.FunType T1) T2) =>
    let _v0 := (eo_to_smt_type T1)
    let _v1 := (eo_to_smt_type T2)
    (smt_typeof_guard _v0 (smt_typeof_guard _v1 (SmtType.FunType _v0 _v1)))
  | (Term.Apply (Term.UOp UserOp.Seq) x1) =>
    let _v0 := (eo_to_smt_type x1)
    (smt_typeof_guard _v0 (SmtType.Seq _v0))
  | (Term.DatatypeType s dd) =>
      if s.startsWith "@" then SmtType.None else SmtType.Datatype s (eo_to_smt_datatype_decl dd)
  ...

def eo_to_smt : Term -> SmtTerm
  | (Term.Boolean b) => (SmtTerm.Boolean b)
  ...
  | (Term.Apply (Term.UOp UserOp.not) x1) => (SmtTerm.not (eo_to_smt x1))
  | (Term.Apply (Term.Apply (Term.UOp UserOp.and) x1) x2) => (SmtTerm.and (eo_to_smt x1) (eo_to_smt x2))
  | (Term.Apply f y) => (SmtTerm.Apply (eo_to_smt f) (eo_to_smt y))
  | (Term.Var (Term.String s) T) => (SmtTerm.Var s (eo_to_smt_type T))
  ...
```

Figure 18: Translation from Eunoia terms and types into SMT.

`(eo_to_smt_datatype_decl dd))`, a type reference `(DatatypeTypeRef s)` to `(SmtType.TypeRef s)`, and constructor and selector terms `DtCons`/`DtSel` to their SMT-LIB counterparts with the translated declaration block. Each of these cases is guarded so that datatype names beginning with the reserved prefix "@" fail to translate; such names are reserved for internal datatypes (e.g. tuples) whose translation is handled specially.

3.3 Satisfiability for Eunoia terms

Figure 19 gives the definition of satisfiability, the specification that the correctness of Logos is stated against. The relation `(smt_satisfiability t b)` holds when `b` is true and some well-formed model evaluates `t` to true, or when `b` is false and every well-formed model evaluates `t` to false. Satisfiability for a Eunoia term (`eo_satisfiability`) is then defined by first translating it to an SMT-LIB term via `eo_to_smt`. Note that a Eunoia term that fails to translate (i.e. maps to `SmtTerm.None`) is neither satisfiable nor unsatisfiable under this definition, as `smt_model_eval` evaluates `None` to `NotValue` in every model. Our top-level correctness theorem will therefore carry explicit translatability assumptions, as described in Section 4.

4 Logos Checker

This section presents the third layer of Logos: the executable proof checker. We first describe the executable Eunoia type system (`eo_typeof`), then the representation of commands and checker state, the execution of commands, and finally the invariants and theorems used to establish the top-level soundness theorem.

4.1 Types of Eunoia Terms

Figure 20 shows selected clauses of `eo_typeof`, the executable method computing the Eunoia type of a Eunoia term (returning `Term.Stuck` on failure). Note that this type system is separate from the SMT-LIB one: it is compiled from the type annotations of the Eunoia signature, and both terms and their types inhabit the single datatype `Term`. Its datatype clauses mirror the SMT-LIB ones, using the same lookup-and-resolve idiom at the Eunoia level: for a constructor term `(DtCons s dd i)`, the helper `eo_dd_resolve` looks up the name `s` in the declaration block `dd` and resolves its type references, after which `eo_typeof_dt_cons_rec` builds the (possibly partially applied) constructor type exactly as in Figure 5. Figure 21 shows a representative theorem relating the two type systems: for terms that translate to well-typed SMT-LIB terms, the translation of the Eunoia type is the SMT-LIB type of the translation.

Figure 22 defines the representation of proofs processed by the checker. A checker state (`CState`) is a stack of objects, each of which is an input assumption (`assume`), a local assumption opened by a scope (`assume_push`), or a proven formula (`proven`); a distinguished `Stuck` state records that checking has failed. A command (`Cmd`) either pushes a local assumption (`assume_push`), checks that the most recently proven formula is an expected one (`check_proven`), or applies a proof rule. Rule applications (`step` and `step_pop`) name a rule of the calculus via the compiled enumeration `CRule` (of which the figure shows a representative sample of the several hundred rules of CPC), a list of terms serving as the rule's explicit arguments (`CArgList`), and a list of stack indices selecting the premises among the previously proven formulas (`CIndexList`). The variant `step_pop` additionally closes the most recently opened local assumption, as used by scope-based reasoning.

Figures 23 and 24 define command execution. The method `eo_state_proven_nth` retrieves the n-th formula of the stack, and the push methods (`eo_push_assume_check`, `eo_push_input_assume_check`, `eo_push_proven_check`) push an object onto the stack guarded by an executable check, moving to `Stuck` when the check fails. In particular, pushing a proven formula requires that it has Eunoia type `Bool` (via `eo_typeof`), and pushing an assumption additionally requires closedness (via `eo_is_closed`). The method `eo_cmd_step_proven` dispatches on the rule name and invokes the corresponding compiled rule program (e.g. `eo_prog_trans`, described below) on the selected arguments and premises, returning the concluded formula or `Stuck`. Command execution (`eo_invoke_cmd`) then folds over the command list (`eo_invoke_cmd_list`). Finally, a command list together with an assumption list constitutes a refutation (`eo_checker_is_refutation`) if, starting from the state built by assuming the input formulas (`eo_invoke_assume_list`), executing the commands yields a state in which `false` has been proven and all local assumptions have been closed (`eo_state_is_refutation`). The relation `eo_is_refutation` lifts this executable check to a proposition.

Figure 25 shows the executable closedness check used by the assumption guards. The method `eo_is_closed_rec` traverses a term with an environment of bound variables, accumulated from the variable lists of `forall` and `exists`; a variable not in the environment witnesses non-closedness. The semantic counterpart of closedness is *stability*: a model can be split into its "global" part (constants and functions) and its variable assignment, and the figure's theorems establish that a closed formula's evaluation depends only on the global part (`smt_model_eval_eq_of_eo_closed`), hence a closed formula that is true remains true in any model agreeing on globals (`stableWhenTrueInAnyVarModel_of_closed`). Stability is what lets the checker's assumptions constrain every model considered during the soundness argument, even as quantifier evaluation re-assigns variables. The guard `assumptionCheckGuard` packages the executable checks performed when an assumption is pushed, as used in the `assume_push` and input-assumption cases of command execution (Figure 24).

Figure 26 defines the translation obligations under which soundness is stated: every input assumption and every explicit rule argument must translate to a well-typed SMT-LIB term. These obligations are collected structurally over the assumption list and the command list. Figure 27 then states the top-level soundness theorem: a successful checker run on translatable inputs implies that the conjunction of the input assumptions is unsatisfiable in the sense of Figure 19. Its proof proceeds by establishing the combined state invariant of Figure 28 for every executed command: the state has its input assumptions as a suffix (shape), every proven formula is true in context (local truth), all assumptions are stable under variable-model changes (stability), all stack formulas have Boolean Eunoia type (typing), and all stack formulas translate (translation).

```
inductive smt_satisfiability : SmtTerm -> Bool -> Prop
  | intro_true  (t : SmtTerm) :
      (exists M : SmtModel, model_wf M /\ (smt_model_eval M t) = (SmtValue.Boolean true)) ->
      smt_satisfiability t true
  | intro_false (t : SmtTerm) :
      (forall M : SmtModel, model_wf M -> (smt_model_eval M t) = (SmtValue.Boolean false))->
      smt_satisfiability t false

def eo_satisfiability (t : Term) (b : Bool) : Prop :=
  (smt_satisfiability (eo_to_smt t) b)
```

Figure 19: Eunoia satisfiability via SMT.

```
def eo_typeof : Term -> Term
  | (Term.Boolean b) => Term.Bool
  | (Term.Var (Term.String s) T) => T
  | (Term.DtCons s dd i) => (eo_typeof_dt_cons_rec (Term.DatatypeType s dd) (eo_dd_resolve s dd) i)
  | (Term.Apply (Term.Apply (Term.UOp UserOp.eq) eo_x1) eo_x2) => (eo_typeof_eq (eo_typeof eo_x1) (eo_typeof eo_x2))
  | (Term.Apply eo_f eo_x) => (eo_typeof_apply (eo_typeof eo_f) (eo_typeof eo_x))
  | ... => ...
```

Figure 20: Selected Eunoia type-synthesis clauses.

```
theorem eo_to_smt_well_typed_and_typeof_implies_smt_type
    (t T : Term) (s : SmtTerm) :
  eo_to_smt t = s ->
  smt_typeof s ≠ SmtType.None ->
  eo_typeof t = T ->
  smt_typeof s = eo_to_smt_type T
```

Figure 21: A representative theorem connecting Eunoia and SMT types.

```
abbrev CIndex := Int

inductive CIndexList : Type where
  | nil : CIndexList
  | cons : CIndex -> CIndexList -> CIndexList
deriving Repr, Inhabited

inductive CArgList : Type where
  | nil : CArgList
  | cons : Term -> CArgList -> CArgList
deriving Repr, Inhabited

inductive CStateObj : Type where
  | assume : Term -> CStateObj
  | assume_push : Term -> CStateObj
  | proven : Term -> CStateObj
deriving Repr, Inhabited

inductive CState : Type where
  | nil : CState
  | cons : CStateObj -> CState -> CState
  | Stuck : CState
deriving Repr, Inhabited

inductive CRule : Type where
  | scope : CRule
  | contra : CRule
  | refl : CRule
  | symm : CRule
  | trans : CRule
  | ...

deriving Repr, Inhabited

inductive Cmd : Type where
  | assume_push : Term -> Cmd
  | check_proven : Term -> Cmd
  | step : CRule -> CArgList -> CIndexList -> Cmd
  | step_pop : CRule -> CArgList -> CIndexList -> Cmd

deriving Repr, Inhabited

inductive CmdList : Type where
  | nil : CmdList
  | cons : Cmd -> CmdList -> CmdList
deriving Repr, Inhabited
```

Figure 22: Checker indices, arguments, states, rules, and commands.

```
def eo_StateObj_proven : CStateObj -> Term
  | (CStateObj.assume F) => F
  | (CStateObj.assume_push F) => F
  | (CStateObj.proven F) => F

def eo_state_proven_nth : CState -> Int -> Term
  | (CState.cons so s), 0 => (eo_StateObj_proven so)
  | (CState.cons so s), n => (eo_state_proven_nth s (n - 1))
  | s, n => (Term.Boolean true)

def eo_state_is_closed : CState -> Bool
  | (CState.cons (CStateObj.assume_push F) s) => false
  | (CState.cons so s) => (eo_state_is_closed s)
  | CState.nil => true
  | s => false

def eo_push_assume_check : Term -> Term -> CState -> CState
  | (Term.Boolean true), F, s => (CState.cons (CStateObj.assume_push F) s)
  | b, F, s => CState.Stuck

def eo_push_input_assume_check : Term -> Term -> CState -> CState
  | (Term.Boolean true), F, s => (CState.cons (CStateObj.assume F) s)
  | b, F, s => CState.Stuck

def eo_push_proven_check : Term -> Term -> CState -> CState
  | (Term.Boolean true), F, s => (CState.cons (CStateObj.proven F) s)
  | b, F, s => CState.Stuck

def eo_push_proven : Term -> CState -> CState
  | F, s => (eo_push_proven_check (eo_is_bool_type F) F s)

def eo_mk_premise_list : Term -> CIndexList -> CState -> Term
  | f, CIndexList.nil, S => (eo_nil f Term.Bool)
  | f, (CIndexList.cons n pl), S => (eo_cons f (eo_state_proven_nth S n) (eo_mk_premise_list f pl S))

def eo_invoke_cmd_check_proven : CState -> Term -> CState
  | (CState.cons (CStateObj.proven F) S), proven => (eo_push_proven_check (eo_eq F proven) F S)
  | S, proven => CState.Stuck
```

Figure 23: Basic checker stack and state operations.

```
def eo_cmd_step_proven (S : CState) : CRule -> CArgList -> CIndexList -> Term
  | CRule.contra, CArgList.nil, (CIndexList.cons n1 (CIndexList.cons n2 CIndexList.nil)) => (eo_prog_contra (Proof.pf (
     eo_state_proven_nth S n1)) (Proof.pf (eo_state_proven_nth S n2)))
  | CRule.refl, (CArgList.cons a1 CArgList.nil), CIndexList.nil => (eo_prog_refl a1)
  | CRule.symm, CArgList.nil, (CIndexList.cons n1 CIndexList.nil) => (eo_prog_symm (Proof.pf (eo_state_proven_nth S n1)))
  | CRule.trans, CArgList.nil, premises => (eo_prog_trans (Proof.pf (eo_mk_premise_list (Term.UOp UserOp.and) premises S)))
  | ... => ...

def eo_cmd_step_pop_proven (S : CState) : CRule -> CArgList -> Term -> CIndexList -> Term
  | CRule.scope, CArgList.nil, A, (CIndexList.cons n1 CIndexList.nil) => (eo_prog_scope A (Proof.pf (eo_state_proven_nth S n1)))
  | ... => ...

def eo_invoke_cmd_step_pop (s : CState) : CState -> CRule -> CArgList -> CIndexList -> CState
  | (CState.cons (CStateObj.assume_push A) s2), r, args, premises => (eo_push_proven (eo_cmd_step_pop_proven s r args A premises) s2)
  | (CState.cons so s2), r, args, premises => (eo_invoke_cmd_step_pop s s2 r args premises)
  | s2, r, args, premises => CState.Stuck

def eo_invoke_cmd : CState -> Cmd -> CState
  | CState.Stuck, c => CState.Stuck
  | S, (Cmd.assume_push proven) =>
     (eo_push_assume_check (eo_and (eo_is_bool_type proven) (eo_is_closed proven)) proven S)
  | S, (Cmd.check_proven proven) => (eo_invoke_cmd_check_proven S proven)
  | S, (Cmd.step r args premises) => (eo_push_proven (eo_cmd_step_proven S r args premises) S)
  | S, (Cmd.step_pop r args premises) => (eo_invoke_cmd_step_pop S S r args premises)

def eo_invoke_cmd_list (S : CState) : CmdList -> CState
  | CmdList.nil => S
  | (CmdList.cons c cmds) => (eo_invoke_cmd_list (eo_invoke_cmd S c) cmds)

def eo_state_is_refutation (s : CState) : Bool :=
  (eo_state_is_closed (eo_invoke_cmd_check_proven s (Term.Boolean false)))

def eo_invoke_assume_list (S : CState) : Term -> CState
  | (Term.Apply (Term.Apply (Term.UOp UserOp.and) F) as) =>
     (eo_push_input_assume_check (eo_and (eo_is_bool_type F) (eo_is_closed F)) F (eo_invoke_assume_list S as))
  | (Term.Boolean true) => S
  | as => CState.Stuck

def eo_checker_is_refutation : Term -> CmdList -> Bool
  | as, cs => (eo_state_is_refutation (eo_invoke_cmd_list (eo_invoke_assume_list CState.nil as) cs))

inductive eo_is_refutation : Term -> CmdList -> Prop
  | intro (F : Term) (c : CmdList) :
     (eo_checker_is_refutation F c) = true -> (eo_is_refutation F c)
```

Figure 24: Checker command execution and refutation recognition.

```
def eo_is_closed_rec : Term -> Term -> Term
  | Term.Stuck, _ => Term.Stuck
  | _, Term.Stuck => Term.Stuck
  | (Term.Var s T), Term.eo_List_nil => (Term.Boolean false)
  | (Term.Var s T), (Term.Apply (Term.Apply Term.eo_List_cons x) vs) =>
      let _v0 := (Term.Var s T)
      (eo_ite (eo_eq _v0 x) (Term.Boolean true) (eo_is_closed_rec _v0 vs))
  | (Term.Apply (Term.Apply (Term.UOp UserOp.forall) vs) x), env =>
      (eo_is_closed_rec x (eo_list_concat Term.eo_List_cons vs env))
  | (Term.Apply (Term.Apply (Term.UOp UserOp.exists) vs) x), env =>
      (eo_is_closed_rec x (eo_list_concat Term.eo_List_cons vs env))
  | (Term.Apply f x), env =>
      (eo_and (eo_is_closed_rec f env) (eo_is_closed_rec x env))
  | (Term.UOp1 u x), env => (eo_is_closed_rec x env)
  | (Term.UOp2 u x x2), env =>
      (eo_and (eo_is_closed_rec x env) (eo_is_closed_rec x2 env))
  | (Term.UOp3 u x x2 x3), env =>
      (eo_and (eo_and (eo_is_closed_rec x env) (eo_is_closed_rec x2 env))
        (eo_is_closed_rec x3 env))
  | x, env => (Term.Boolean true)

def eo_is_closed : Term -> Term
  | Term.Stuck => Term.Stuck
  | t => (eo_is_closed_rec t Term.eo_List_nil)

def assumptionCheckGuard (A : Term) : Term :=
  eo_and (eo_is_bool_type A) (eo_is_closed A)

/-- Two models agree on the global part of the interpretation. -/
structure model_agrees_on_globals (M N : SmtModel) : Prop where
  consts : ∀ s T, model_lookup M false s T = model_lookup N false s T
  funs : ∀ fid T U, model_fun_lookup M fid T U = model_fun_lookup N fid T U
  usortDom : M.usortDom = N.usortDom

theorem smt_model_eval_eq_of_eo_closed
    (P : Term) (hClosed : eo_is_closed P = Term.Boolean true)
    (M N : SmtModel) (hAgree : model_agrees_on_globals M N) :
  smt_model_eval M (eo_to_smt P) = smt_model_eval N (eo_to_smt P)

/-- A formula remains true when only SMT variable assignments are changed. -/
def StableInAnyVarModel (M : SmtModel) (P : Term) : Prop :=
  ∀ N, model_wf N -> model_agrees_on_globals M N ->
    eo_interprets N P true

/-- A formula remains true under variable-model changes whenever it is true. -/
def StableWhenTrueInAnyVarModel (P : Term) : Prop :=
  ∀ M, model_wf M -> eo_interprets M P true ->
    StableInAnyVarModel M P

theorem stableWhenTrueInAnyVarModel_of_closed
    (P : Term) (hClosed : eo_is_closed P = Term.Boolean true) :
  StableWhenTrueInAnyVarModel P
```

Figure 25: The executable EO closedness check used by assumption guards, and the key theorem showing that closed formulas are stable under changes to SMT variable assignments.

```
def eoHasSmtTranslation (t : Term) : Prop :=
  smt_typeof (eo_to_smt t) ≠ SmtType.None

def cArgListTranslationOk : CArgList -> Prop
  | CArgList.nil => True
  | CArgList.cons a args => eoHasSmtTranslation a ∧ cArgListTranslationOk args

def cmdTranslationOk : Cmd -> Prop
  | Cmd.assume_push A => eoHasSmtTranslation A
  | Cmd.step _ args _ => cArgListTranslationOk args
  | _ => True

inductive TranslatableAssumptionList : Term -> Prop
  | base : TranslatableAssumptionList (Term.Boolean true)
  | step (A rest : Term) :
      eoHasSmtTranslation A ->
      TranslatableAssumptionList rest ->
      TranslatableAssumptionList (Term.Apply (Term.Apply (Term.UOp UserOp.and) A) rest)

inductive CmdListTranslationOk : CmdList -> Prop
  | nil : CmdListTranslationOk CmdList.nil
  | cons (c : Cmd) (cs : CmdList) :
      cmdTranslationOk c ->
      CmdListTranslationOk cs ->
      CmdListTranslationOk (CmdList.cons c cs)
```

Figure 26: Translation obligations for checker assumptions and commands.

```
theorem correct_eo_is_refutation (F : Term) (pf : CmdList) :
  TranslatableAssumptionList F ->
  CmdListTranslationOk pf ->
  (eo_is_refutation F pf) ->
  (eo_satisfiability F false)
```

Figure 27: Top-level checker soundness theorem.

```
structure ContextualTruth
    (M : SmtModel) (assumes pushes P : Term) : Prop where
  true_here :
    eo_interprets M assumes true ->
    eo_interprets M pushes true ->
    eo_interprets M P true
  true_in_var_model :
    forall N, model_wf N ->
      model_agrees_on_globals M N ->
      eo_interprets N assumes true ->
      eo_interprets N pushes true ->
      eo_interprets N P true

def checkerLocalTruthInvariant (M : SmtModel) : CState -> Prop
  | CState.nil => True
  | CState.cons (CStateObj.proven P) s =>
      ContextualTruth M (stateAssumes s) (statePushes s) P ∧
      checkerLocalTruthInvariant M s
  | CState.cons _ s => checkerLocalTruthInvariant M s
  | CState.Stuck => True

def checkerAssumptionStabilityInvariant (M : SmtModel) : CState -> Prop
  | CState.nil => True
  | CState.cons (CStateObj.assume A) s =>
      StableWhenTrueInAnyVarModel A ∧ checkerAssumptionStabilityInvariant M s
  | CState.cons (CStateObj.assume_push A) s =>
      StableWhenTrueInAnyVarModel A ∧ checkerAssumptionStabilityInvariant M s
  | CState.cons (CStateObj.proven _) s =>
      checkerAssumptionStabilityInvariant M s
  | CState.Stuck => True

def checkerShapeInvariant : CState -> Prop
  | CState.Stuck => True
  | s => stateAssumptionSuffix s

def checkerStateInvariant (M : SmtModel) (s : CState) : Prop :=
  checkerShapeInvariant s ∧
  checkerLocalTruthInvariant M s ∧
  checkerAssumptionStabilityInvariant M s ∧
  checkerTypeInvariant s ∧
  checkerTranslationInvariant s
```

Figure 28: The combined checker invariant, separating local contextual truth from assumption stability alongside the shape, type, and translation invariants.

4.2 Rule Correctness

The soundness proof is modularized per proof rule. Figure 29 shows the interface. For each rule, a local theorem (one per rule, e.g. Figure 31 for `trans`) packages the rule's correctness into `StepRuleProperties`: whenever the premises hold (as `RulePremiseEvidence`, which supplies their truth both in the current model and in any model agreeing on globals), the conclusion is true, and the conclusion translates to SMT. The dispatching theorem `cmd_step_proven_facts_of_invariants` in `RuleLemmas.lean` consumes these per-rule theorems to establish `CmdStepFacts` for an arbitrary `Cmd.step`, which is exactly what the invariant-preservation argument of the previous subsection requires. Consequently, adding or reproving a rule only requires providing its local theorem; the checker-level soundness proof is unchanged.

Figure 30 shows a representative compiled rule program, for the transitivity rule `trans`. Rule programs manipulate the `Proof` wrapper, whose payload is the (conjunction of) premise formulas assembled by `eo_mk_premise_list` in Figure 23; the helper `eo_requires` implements the Eunoia `eo::requires` side condition, comparing terms syntactically (via `teq`) and propagating `Stuck`. The program `eo_prog_trans` chains the premise equalities `t1 = t2`, `t2 = t3`, ... and concludes the equality between the endpoints.

```
structure RulePremiseEvidence
    (M : SmtModel) (premises : List Term) : Prop where
  true_here :
    AllInterpretedTrue M premises
  true_in_var_model :
    forall N, model_wf N ->
      model_agrees_on_globals M N ->
      AllInterpretedTrue N premises

structure StepRuleProperties
    (M : SmtModel) (premises : List Term) (P : Term) : Prop where
  facts_of_true :
    RulePremiseEvidence M premises ->
    eo_interprets M P true
  has_smt_translation :
    eoHasSmtTranslation P

structure CmdStepFacts (M : SmtModel) (s : CState) (P : Term) : Prop where
  true_of_context :
    eo_interprets M (stateAssumes s) true ->
    eo_interprets M (statePushes s) true ->
    eo_interprets M P true
  true_in_var_model :
    forall N, model_wf N ->
      model_agrees_on_globals M N ->
      eo_interprets N (stateAssumes s) true ->
      eo_interprets N (statePushes s) true ->
      eo_interprets N P true
  has_smt_translation :
    eoHasSmtTranslation P

theorem cmd_step_proven_facts_of_invariants
    (M : SmtModel) (hM : model_wf M)
    (s : CState) (_hNotStuck : s ≠ CState.Stuck)
    (r : CRule) (args : CArgList) (premises : CIndexList) :
  checkerLocalTruthInvariant M s ->
  checkerAssumptionStabilityInvariant M s ->
  checkerTypeInvariant s ->
  checkerTranslationInvariant s ->
  cmdTranslationOk (Cmd.step r args premises) ->
  eo_typeof (eo_cmd_step_proven s r args premises) = Term.Bool ->
  CmdStepFacts M s (eo_cmd_step_proven s r args premises)
```

Figure 29: The step-only rule-correctness theorem in `RuleLemmas.lean`; plain `Cmd.step` reasoning uses local truth plus assumption stability, and rule proofs receive contextual premise evidence that is stable across variable-model changes.

```
inductive Proof : Type where
  | pf : Term -> Proof
  | Stuck : Proof

def eo_requires : Term -> Term -> Term -> Term
  | x1, x2, x3 =>
      (if (teq x1 x2) then
        (if (!(teq x1 Term.Stuck)) then x3 else Term.Stuck)
        else Term.Stuck)

def mk_trans : Term -> Term -> Term -> Term
  | Term.Stuck, _, _ => Term.Stuck
  | _, Term.Stuck, _ => Term.Stuck
  | t1, t2,
      (Term.Apply
        (Term.Apply (Term.UOp UserOp.and)
          (Term.Apply (Term.Apply (Term.UOp UserOp.eq) t3) t4))
        tail) =>
      (eo_requires t2 t3 (mk_trans t1 t4 tail))
  | t1, t2, (Term.Boolean true) =>
      (Term.Apply (Term.Apply (Term.UOp UserOp.eq) t1) t2)
  | _, _, _ => Term.Stuck

def eo_prog_trans : Proof -> Term
  | (Proof.pf
      (Term.Apply
        (Term.Apply (Term.UOp UserOp.and)
          (Term.Apply (Term.Apply (Term.UOp UserOp.eq) t1) t2))
        tail)) =>
      (mk_trans t1 t2 tail)
  | _ => Term.Stuck
```

Figure 30: The `Logos.lean` definition of the `trans` rule program and its immediate dependencies.

```
theorem cmd_step_trans_properties
    (M : SmtModel) (hM : model_wf M)
    (s : CState) (args : CArgList) (premises : CIndexList) :
  cmdTranslationOk (Cmd.step CRule.trans args premises) ->
  AllHaveBoolType (premiseTermList s premises) ->
  eo_typeof (eo_cmd_step_proven s CRule.trans args premises) = Term.Bool ->
  StepRuleProperties M (premiseTermList s premises)
    (eo_cmd_step_proven s CRule.trans args premises)
```

Figure 31: An example rule proof: the `trans` step packages contextual premise evidence and SMT translatability into `StepRuleProperties`.